Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «РОБОТОТЕХНИКА И КОМПЛЕКСНАЯ АВТОМАТИЗАЦИЯ»

КАФЕДРА РК-9 «КОМПЬЮТЕРНЫЕ СИСТЕМЫ АВТОМАТИЗАЦИИ ПРОИЗВОДСТВА»

РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

К ВЫПУСКНОЙ КВАЛИФИКАЦИОННОЙ РАБОТЕ

НА ТЕМУ:
«Облачная система контроля положения штабелера автоматизированного склада»

Студент ____РК9-82Б____ _____ А. Ж. Сарсенбеков
(Группа) (Подпись, дата) (И.О.Фамилия)

Руководитель ВКР _____ _С. А. Буханов____
(Подпись, дата) (И.О.Фамилия)

Нормоконтролер _____ __И.А. Мещихин____
(Подпись, дата) (И.О.Фамилия)

2023 г.

РЕФЕРАТ

Выпускная квалификационная работа выполнена по теме: «Облачная система контроля положения штабелера автоматизированного склада».

Расчетно-пояснительная записка содержит 50 листов формата А4, 27 рисунков, 15 используемых источников, 4 приложения.

Целью выполнения работы является создание облачной системы контроля положения штабелера автоматизированного склада. В работе приведен анализ существующей системы, выявлены его недостатки. Разработана функциональная модель облачной системы контроля положения штабелера. Выбрана облачная платформа. Разработаны алгоритмы и исходные коды устройства контроля положения штабелера; разработан исходный код обработчика облачной платформы, разработан графический интерфейс мобильного приложения; принципиальная электрическая схема подключения устройства.

Объектом исследования является автоматический склад в составе ГПС производства фирмы Denford, находящийся в лабораторной аудитории 297 кафедры РК9 «Компьютерные системы автоматизации производства» МГТУ им. Н. Э. Баумана.

В заключении описаны результаты проделанной работы.

Ключевые слова: система контроля, облачная платформа, интернет вещей, промышленный интернет вещей, Rightech IoT Cloud.

СОДЕРЖАНИЕ

РЕФЕРАТ
СОДЕРЖАНИЕ
СПИСОК ОБОЗНАЧЕНИЙ И СОКРАЩЕНИЙ
ВВЕДЕНИЕ
1. Предпроектное обследование объекта автоматизации
1.1. Целевое обследование объекта автоматизации
1.2. Недостатки существующей системы
2. Концептуальное проектирование
2.1. Постановка задачи
2.2. Разработка функциональной схемы системы контроля положения штабелера
3. Техническое задание
3.1 Общие сведения
3.2. Назначение и цели создания системы
3.3. Требования к системе в целом
4. Техническое проектирование
4.1. Выбор облачной платформы
4.2. Выбор микроконтроллера
4.3. Выбор дальномера
4.4. Взаимодействие между устройством и облачной платформой Rightech IoT Cloud
5. Рабочее проектирование
5.1. Принципиальная схема подключения ESP-WROOM-32 к дальномерам
5.2. Разработка алгоритма работы устройства контроля положения
5.3. Настройка облачной платформы Rightech IoT Cloud
6. Апробирование
ЗАКЛЮЧЕНИЕ
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ
Приложение А
Приложение Б
Приложение В
Приложение Г

СПИСОК ОБОЗНАЧЕНИЙ И СОКРАЩЕНИЙ

«ГПС» - это производственная система, которая легко адаптируется к изменениям типа и количества производимого продукта. Машины и компьютеризированные системы могут быть сконфигурированы для производства различных деталей и управления меняющимися уровнями производства.

«ПЛК» («Программируемые логические контроллеры») — технические средства, используемые для автоматизации технологических процессов. Это электронное специализированное устройство, работающее в реальном масштабе времени.

«IOT» («Internet of Things») – система взаимосвязанных устройств (вещей), оснащенных средствами для беспроводной передачи данных между друг другом, внешней средой без непосредственного вмешательства человека.

«IIOT» («Industrial Internet of Things») – взаимосвязанные датчики, исполнительные устройства, промышленное оборудование и другие девайсы, объединенные в сеть совместно с промышленным программным обеспечением для управления производством.

«M2M» («Machine-to-Machine») – общее название технологий, которые позволяют машинам обмениваться информацией друг с другом, или же передавать её в одностороннем порядке.

«RIC» («Rightech IoT Cloud» – облачная платформа от российской компании «КОМНЭТ», используемая в данной работе.

«MQTT» («Message Queuing Telemetry Transport») – упрощённый сетевой протокол, работающий поверх TCP/IP, ориентированный на обмен сообщений между устройствами по принципу издатель-подписчик.

«ESP 32» – микроконтроллер, представляющей из себя систему на кристалле с интегрированными Wi-Fi и Bluetooth контроллерами.

«GPIO» – это сигнальный контакт на интегральной схеме или плате, который можно использовать для выполнения функций цифрового ввода или вывода.

«Модель» – функциональное описание класса подключаемых устройств.

«Объект» – виртуальный аналог конкретного подключаемого устройства, отражает состояние устройства и возможности управления.

ВВЕДЕНИЕ

В данный момент мир находится в состоянии четвертой промышленной революции, которая приводит к изменениям не только в экономической, социальной, политической сферах жизни людей, но и в промышленности. Одним из сторон данного технологического прогресса является использование кибер-физических систем и технологии интернета вещей («IoT») в промышленности.

Кибер-физическая система – это информационно-технологическая концепция, включающая в себя интеграцию вычислительных ресурсов в физическую сущность любого вида [1]. Внедрение кибер-физических систем в производство принято называть индустрия 4.0, основная цель которой – максимальная автоматизация производства, частичное или полное исключение человека из производственных процессов. Такие системы позволяют улучшить их, обеспечивая обмен данными между промышленным оборудованием в режиме реального времени, который возможен только в локальной сети, например, в пределах одной ГПЯ.

Для того чтобы создать умное производство, необходимо также применять технологии интернета вещей, которые позволяют в автоматическом режиме вести мониторинг состояния оборудования, контролировать производственный процесс не только в рамках одного производственного модуля, но и в рамках всего предприятия в целом [2].

Термин «интернет вещей» был впервые использован Кевином Энтоном, основателем исследовательской группы Auto-ID при Массачусетском технологическом университете (MIT), во время презентации для руководства компании Procter&Gamble в 1999 году [3]. С тех пор было представлено множество определений данного понятия, но в настоящее время общепринятым является следующее: интернет вещей (Internet of Things) – система взаимосвязанных устройств (вещей), оснащенных средствами для

беспроводной передачи данных между друг другом, внешней средой без непосредственного вмешательства человека. Развитие данной технологии привело к появлению такого направления, как интернет всего (Internet of Everything) – взаимодействие между людьми, вещами, данными и процессами, объединенное в общую систему, цель которой – помочь пользователям принимать оптимальные решения. Кроме этого, существует отдельное направление – промышленный интернет вещей (Industrial Internet of Things) – взаимосвязанные датчики, исполнительные устройства, промышленное оборудование и другие девайсы, объединенные в сеть совместно с промышленным программным обеспечением для управления производством. Эта взаимосвязь позволяет обмениваться данным между устройствами (M2M взаимодействие), анализировать их и вносить изменения в производственный процесс для получения наибольшей производительности. Данная технология позволяет реализовать концепцию умного предприятия в рамках индустрии 4.0 [4].

Внедрение облачных технологий промышленного интернета вещей в производство приводит к переходу от классической структуры АСУ ТП, представленной в виде пирамиды и состоящей из четырех уровней:

1) уровень планирования ресурсов (ERP);
2) уровень управления процессами (SCADA);
3) уровень контроля (ПЛК, HMI);
4) физический уровень (датчики, приводы); к структуре, в которой планирование ресурсов предприятия, управление процессами и контроль осуществляется в облачной платформе.

Основными элементами облачных технологий промышленного интернета вещей являются (рисунок 1):

1) облачные вычисления;
2) высокоскоростная сеть реального времени;
3) физический уровень (датчики, приводы).

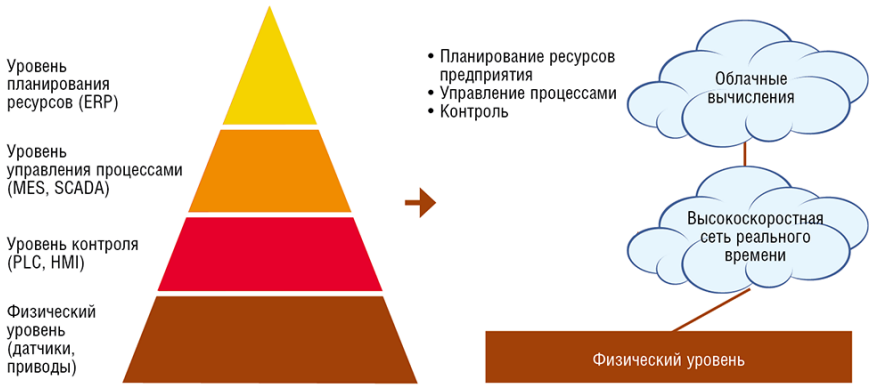


Рисунок 1 – Переход от традиционных АСУ ТП к IIOT

Применение данных технологий позволяет легко масштабировать существующие производственные участки, производство в целом, использовать данные, полученные с датчиков оборудования для оценки его остаточного ресурса, а также для предотвращения нештатных ситуаций, приводящих к необратимым отказам оборудования.

1. Предпроектное обследование объекта автоматизации

1.1. Целевое обследование объекта автоматизации

Объектом исследования является ГПС производства фирмы Denford, а именно один из модулей ее транспортно-накопительной системы – автоматизированный склад, который находится в лабораторной аудитории 297 кафедры РК9 «Компьютерные системы автоматизации производства» МГТУ им. Н. Э. Баумана. Структурная схема ГПС Denford приведена на рисунке 2.

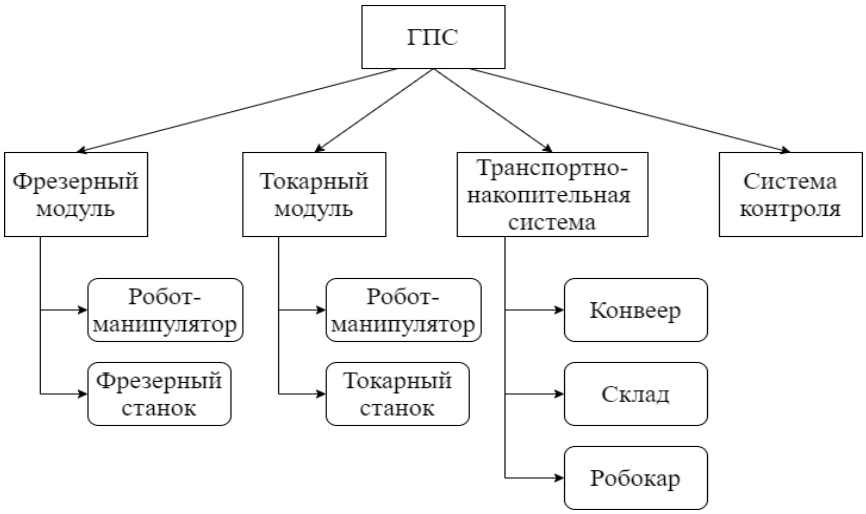


Рисунок 2 – Структурная схема ГПС Denford

Автоматизированный склад представляет собой портальный манипулятор, который управляется ПЛК NextMove ESB-2 от компании ABB. Привод манипулятора электрический, рабочий элемент склада (штабелер) перемещается по трем направлениям декартовой системы координат с помощью трех сервоприводов MicroFlex также производства компании ABB, каждый из которых состоит из шагового двигателя, оптического энкодера и блока управления. Для предотвращения аварийных ситуаций на концах каждой

из осей предусмотрены концевые выключатели. Структурная схема автоматизированного склада представлена на рисунке 3.

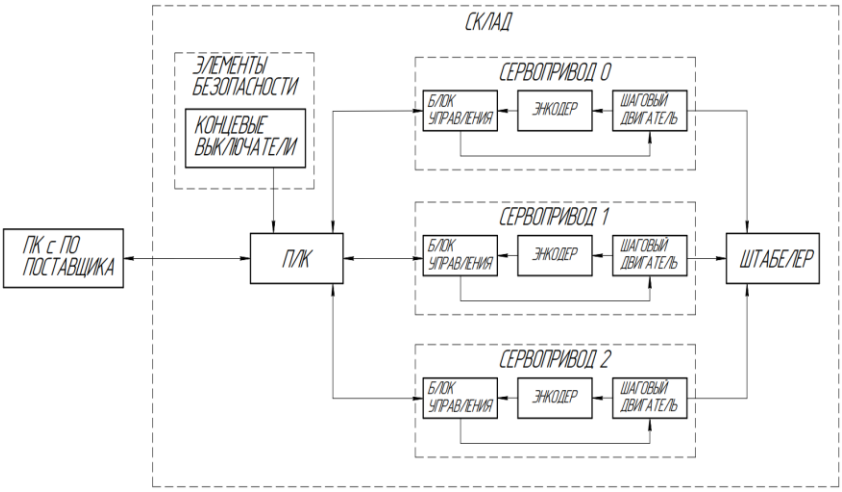


Рисунок 3 – Структурная схема автоматизированного склада

Основными функциями складского модуля являются хранение паллет с заготовками (готовыми изделиями) и их перемещение между стеллажом склада и конвейером. Хранение паллет на складе устроено следующим образом: стеллаж склада имеет 50 ячеек, в которых могут располагаться паллеты, однако из-за конструктивных особенностей штабелера (он не поворотный), единовременно может храниться до 25 паллет с заготовками на правой половине стеллажа и до 25 паллет с готовыми изделиями на левой половине стеллажа соответственно. Изображение автоматизированного склада представлено на рисунке 4.

Рисунок 4 – Автоматизированный склад в лабораторной аудитории 297 кафедры РК9 «Компьютерные системы автоматизации производства» МГТУ им. Н. Э. Баумана.

Наибольший интерес при анализе объекта автоматизации (склада) представляет функция перемещения паллет между конвейером и стеллажом склада. Для описания функции перемещения паллет существующей системы строится функциональная диаграмма IDEF0 «as is» [5] (рисунок 5).

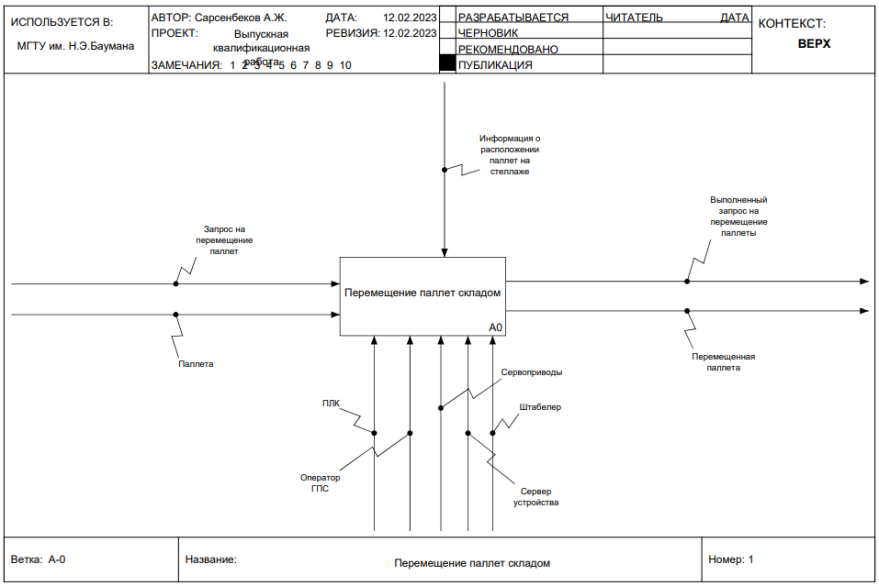


Рисунок 5 – Функциональная модель перемещения паллеты складом «as is» (блок А-0)

Данная функциональная диаграмма отражает процесс перемещения паллеты в рамках существующей системы от момента получения заявки на перемещение паллеты оператором ГПС (например, в роли заявки может выступать маршрутная карта) до фактической транспортировки паллеты в определенное положение (оно также определяется согласно маршрутной карте) (рисунок 6). На первом этапе оператору поступает запрос на перемещение паллеты, после чего он определяет расположение (номер и стеллаж) необходимой ячейки. Затем оператору необходимо ввести в диалоговое окно корректную команду с координатами ячейки. И если введенные данные правильные, то вызывается функция из библиотеки MML, после чего ПЛК обрабатывает ее, передает управляющую команду на сервоприводы, и происходит перемещение паллеты.

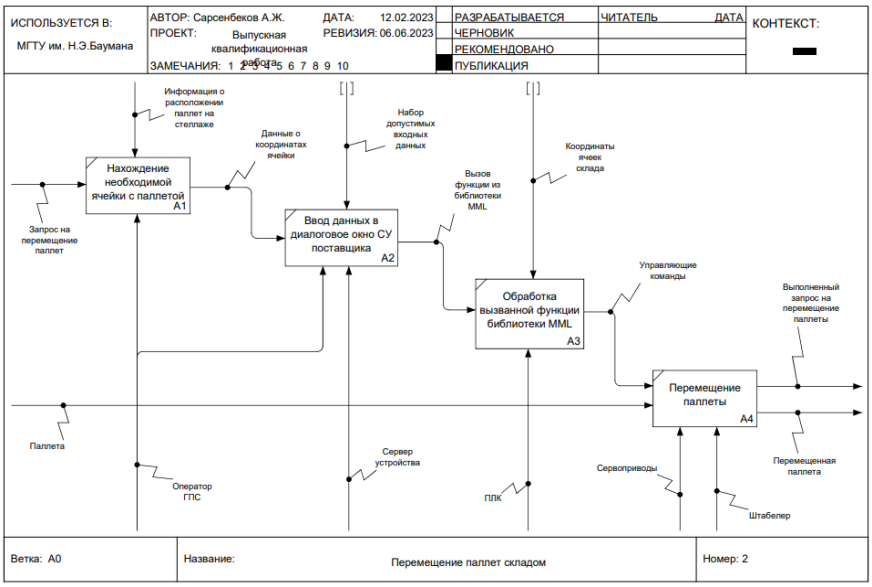


Рисунок 6 – Функциональная модель перемещения паллеты складом «as is» (блок A0)

1.2. Недостатки существующей системы

В исследуемой системе управление складом осуществляется непосредственно оператором с помощью программного обеспечения производителя, установленного на сервере устройства (в роли которого выступает ПК с операционной системой Windows XP), подключенном к ПЛК через интерфейс USB. Данная система управления имеет ряд существенных недостатков:

- отсутствует режим автоматической работы склада;
- отсутствует возможность масштабируемости системы (внедрение нового модуля в состав ГПС влечет за собой полное изменение существующей архитектуры системы);
- ограниченный функционал (существует только один режим перемещения между «выученными» положениями);

- отсутствует система контроля положения штабелера

Одним из путей устранения данных недостатков является внедрение облачной системы управления складом. Данная система позволит решить ключевую проблему, актуальную для большинства современных ГПС – отсутствие масштабируемости. Проблема существует из-за того, что на данный момент стандарты, регламентирующие протоколы взаимодействия между устройствами различных производителей находятся в разработке. Использование облачных технологий решает данную проблему, потому что в такой системе устройства не взаимодействуют друг с другом непосредственно, они взаимодействуют с облачной платформой.

Одной из больших проблем является отсутствие системы контроля положения штабелера. В установленных на складе сервоприводах применяются инкрементальные энкодеры, которые после отключения питания обнуляют свое положение в памяти ПЛК, при этом штабелер физически остается на месте, поэтому сразу после включения склада его необходимо предварительно вывести в нулевое положение.

В рамках выпускной квалификационной работы было принято решение разработать систему контроля положения штабелера автоматизированного склада, которая будет обрабатывать полученные данные с датчиков облачной системой.

2. Концептуальное проектирование

2.1. Постановка задачи

Развитие облачных технологий - ключевое направление в Интернете вещей и представляет огромный потенциал для промышленности. Одним из главных преимуществ таких систем является упрощение интеграции новых устройств в производственные ячейки. Благодаря наличию быстрого канала связи и мощных вычислительных ресурсов, предоставляемых облачной платформой, возможны решения задач, которые ранее были недоступны. Например, собирать большие объемы данных с датчиков на машинах для оценки их состояния.

Разработка облачной системы контроля положения штабелера с учетом требований, полученных в результате целевого обследования объекта автоматизации, создаст условия для дальнейшей интеграции накопительного модуля в состав ГПС, что в свою очередь позволит автоматизировать перемещение паллет между конвейером и стеллажом склада. При этом будет решена проблема взаимодействия между оборудованием различных производителей, потому что, как упоминалось ранее, взаимодействие между модулями ГПС будет происходить с помощью облачной платформы. На основании вышесказанного была сформулирована цель выпускной квалификационной работы.

Цель работы: разработать систему контроля положения штабелера складской системы в составе гибкой производственной системы, которая взаимодействует с облачной системой управления.

В ходе работы над проектом были поставлены следующие **задачи**:

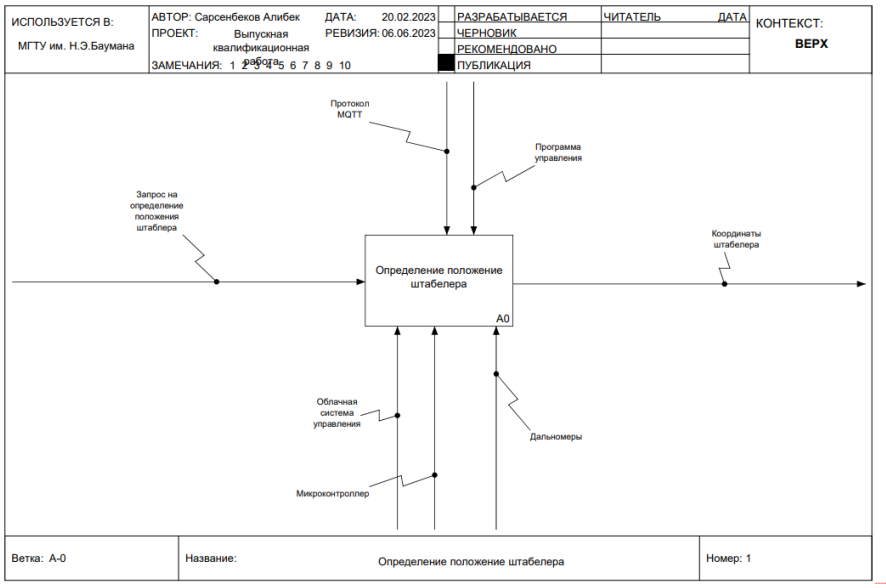
1) проанализировать существующую систему управления складской системой и выявить ее недостатки;

2) на основе анализа полученных данных сформулировать требования, предъявляемые к разрабатываемой системе;
3) разработать функциональную модель работы системы контроля положения штабелера;
4) подобрать облачную платформу для обработки результатов измерений;
5) разработать устройство контроля положения штабелера;
6) разработать алгоритмы основных функций устройства;
7) разработать исходный код программной части облачной системы контроля положения штабелера;
8) разработать интерфейс приложения, который будет показывать актуальное положение штабелера.

Примечание [BSA1]: принципиальную схему основной аппаратной части устройства

2.2. Разработка функциональной схемы системы контроля положения штабелера

В ходе работы над проектом была разработана функциональная модель создаваемой системы распознавания местоположения штабелера (рисунок 7).



Примечание [BSA2]: программа управления подается снизу

Рисунок 7 – Функциональная модель системы контроля положения штабелера (блок А-0)

Функциональная модель системы отражает этапы взаимодействия облачной системы управления с микроконтроллером, который производит измерение расстояния (рисунок 8).

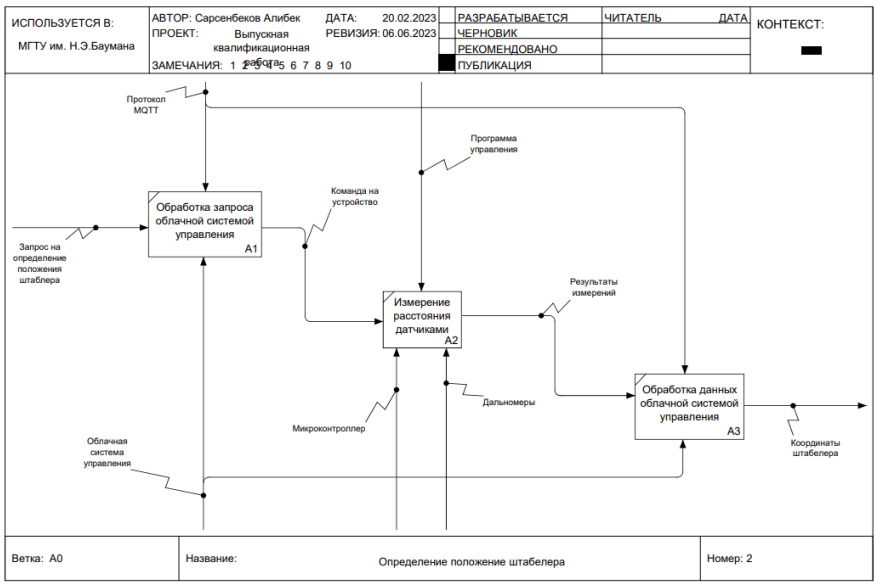


Рисунок 8 – Функциональная модель системы контроля положения штабелера (блок А0)

На первом этапе на облачную систему поступает запрос на определение местоположение штабелера. После чего облачная система обрабатывает запрос, подключается к микроконтроллеру по протоколу MQTT и передает последнему команду на измерение расстояния. Микроконтроллер считывает данные с дальномеров и отправляет облачной системе результаты измерений также по протоколу MQTT. Облачная система управления обрабатывает полученные данные и выдает на выходе координаты штабелера.

3. Техническое задание

3.1 Общие сведения

Полное наименование системы и ее условное обозначение

Облачная система контроля положения штабелера автоматизированного склада

Наименование предприятий (объединений) разработчика и заказчика (пользователя) системы и их реквизиты

Разработчик:

Сарсенбеков Алибек Жанатулы

Предприятие:

Группа РК9-82Б

Юридический адрес: г. Москва, Измайловский пр-т, 75/1.

Заказчик: МГТУ им. Н.Э. Баумана

Юридический адрес: г. Москва, 2-я Бауманская улица, 5 стр.1

Плановые сроки работы по созданию системы

Дата начала работы – 07.09.2022 г.

Дата окончания работы – 29.04.2023 г.

3.2. Назначение и цели создания системы

Назначение системы

Разрабатываемая облачная система предназначена для контролем положением штабелера автоматизированного склада

Цели создания системы

Облачная система управления складом в составе гибкой производственной системы позволит решать следующие задачи:

- дальнейшая интеграция склада в состав гибкой производственной системы;
- определение положения штабелера без необходимости выведения его в нулевую точку;
- обмен данными между облачной платформой, складом и другими объектами гибкой производственной системы для предотвращения нештатных ситуаций;
- сокращение времени непроизводительной работы оборудования за счет исключения необходимости перемещения штабелера в нулевую точку.

3.3. Требования к системе в целом

Требования к функционированию системы:

- система должна определять положение штабелера;
- система должна взаимодействовать с облачной платформой;
- в случае выявления ошибок во время определения положения, система должна выдавать данные об ошибке на облачную платформу;
- программная часть системы должна иметь открытый исходный код, позволяющий другим пользователям изменять его в зависимости от их целей;
- система должна иметь приложение, в котором высвечивается координаты штабелера, номер ячейки либо положение (робокар, нулевая точка, конвейер)
- интерфейс приложения должен быть интуитивно понятен пользователю.

Требования к структуре системы:

Структура облачной системы контроля положением штабелера состоит из двух уровней:

- нижний уровень состоит из устройства измерения координат положения штабелера
- верхний уровень состоит из облачной платформы, которая обрабатывает данные полученные устройством измерением координат положения штабелера и выводит результаты.

Техническое задание разработано в соответствии с ГОСТ 34.602-89 [6].

4. Техническое проектирование

4.1. Выбор облачной платформы

Облачные платформы – платформы для облачных вычислений, представляющие собой готовое программное и аппаратное обеспечение, сдающееся в аренду через Интернет для развертывания, разработки, тестирования своих приложений.

Идея создания облачных платформ была сформирована еще во времена появления Интернета, однако первая облачная платформа появилась только в 2006 году под названием «Amazon Web Services» (хотя предоставлять доступ к вычислительным ресурсам через Интернет компания начала еще в 2002 году). Именно в 2006 Amazon предложила более 50 различных услуг в 14 географических регионах.

Следующей крупной платформой стала система Microsoft Azure, появившаяся в 2010 (сейчас занимает второе почетное место по количеству пользователей и предоставляемых услуг после Amazon). В 2011 в мире был представлен третий крупный игрок на рынке облачных вычислений - Google Cloud Platform. Через год после этого началось активное создание новых платформ и сервисов от различных компаний, и как следствие снижение цен на аренду платформ. Были созданы такие сервисы как: i-Teco OpenStack Cloud, Cloud OS от Azure, Jelastic для PHP и Java приложений и другие.

Сегодня облачные платформы как никогда пользуются популярностью. Основными преимуществами использования облачных платформ является быстрота создания новых приложений, гибкость и масштабируемость системы [7].

В рамках выпускной квалификационной работы была выбрана облачная платформа Rightech IoT Cloud от компании КОМНЭТ. Rightech IoT Cloud —

это бескодовая (no-code) IoT-платформа для быстрого создания прикладных проектов интернета вещей.

Программный продукт Rightech IoT Cloud (RIC, рус. Райтек ИоТ Клауд) от компании-разработчика КОМНЭТ является фреймворк-инструментом и предназначен для быстрого создания разработчиками приложений интернета вещей. Платформа RIC реализована на принципах универсализации и имеет низкую зависимость от конкретного оборудования и протоколов, что позволяет легко объединять разнородные устройства в едином разрабатываемом на платформе решении. Применяя инструменты платформы Rightech разработчики могут создавать IoT-решения с малым количеством кода и с использованием готовых компонентов, повторно используя до 90% решений и запуская аналогичные проекты [8].

Основным преимуществом RIC является бесплатный доступ к базовому функционалу платформы, которые в рамках выпускной квалификационной работы решают поставленные задачи.

Реализация любого решения на платформе RIC включает в себя ряд этапов:

1. Проектирование модели
2. Создание объект
3. Подключение устройства
4. * Построение сценариев автоматизации
5. * Постобработка данных
6. * Использование вспомогательных сущностей
7. * Реализация взаимодействия с платформой посредством REST API

* — необязательные этапы

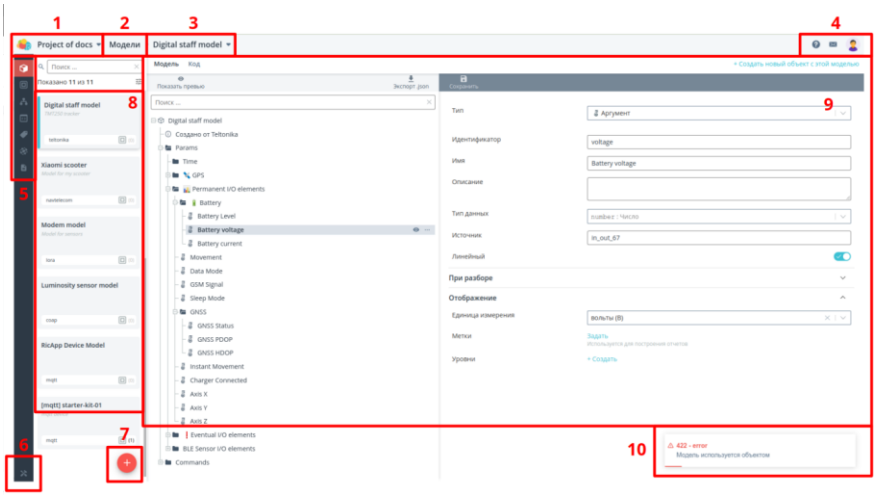


Рисунок 9 – Интерфейс облачной платформы RIC

Примечание [BSA3]: на все рисунки должны быть ссылки из текста

1 — выбранный проект

2 — выбранная сущность

3 — название сущности

4 — разделы Информация, Сообщения, Личный кабинет

5 — выбор сущности

6 — раздел Администрирование проекта

7 — добавление карточки сущности

8 — область с карточками

9 — динамический интерфейс, зависящий от выбранной сущности

10 — код и текст возникшей ошибки

4.2. Выбор микроконтроллера

Основополагающей идеей развития микроконтроллеров на сегодняшний день остается минимизация размеров кристаллов микросхем с одновременным увеличением содержащихся в единой структуре интегрированных компонентов. Добившись значительного успеха с чипами ESP8266, компания Espressif Systems не остановилась на достигнутом. Продолжив движение в ногу со временем и сотворив маленькое чудо, она выпустила на рынок совершенно новое, мощное и гибкое решение, связывающее в себе не только популярные стандарты беспроводной передачи информации Wi-Fi и Bluetooth, встречающиеся повсеместно практически в любых современных электронных устройствах, но и по-настоящему обширные инновационные возможности, расширяющие спектр использования микроконтроллера ESP32 в разнообразных проектах с низким энергопотреблением.

Контроллеры с ядром ESP32 могут быть задействованы в различных и по-своему уникальных приложениях: связующие устройства для концептуальных вычислительных сетей Интернет вещей, включающие инструменты по сбору и хранению данных или объединению всевозможных датчиков, устройства аудио или видео стриминга, дополнительные надстройки к полноценным электронным продуктам, элементы механизмов распознавания речи или изображений. ESP32 может участвовать при разработке схем координирования и взаимодействия индустриального оборудования или систем мониторинга за электронными метками, образовательной, промышленной и сервисной робототехнике, в детских игрушках, в любых компактных портативных умных устройствах [9]. Изображение микроконтроллера приведено на рисунке 10.

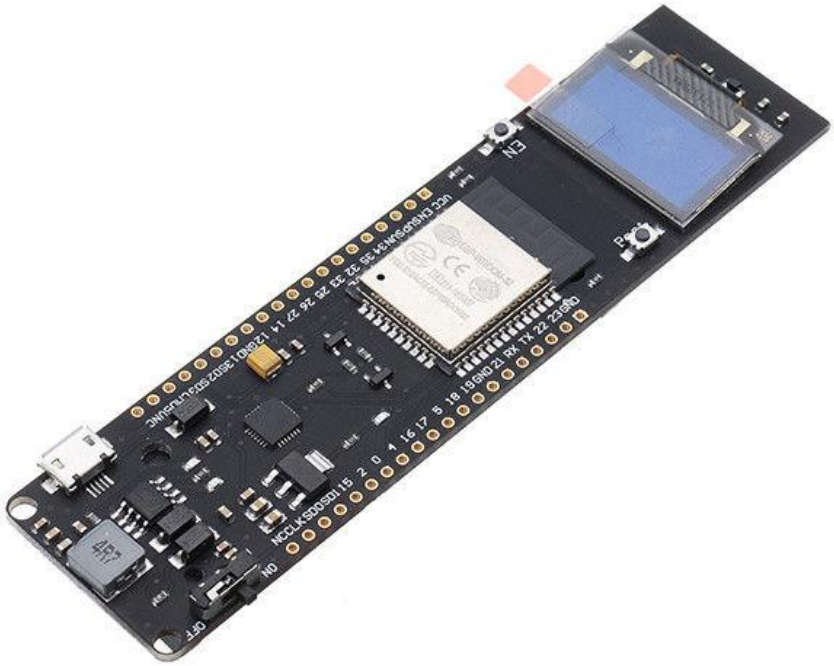


Рисунок 10 – Wemos TTGO WiFi/Bluetooth BLE с OLED-экраном 0.96 дюйма на контроллере ESP-WROOM-32 ESP32

Технические характеристики [8]:

- Питание: 5 В / 3.7 В от внешней батареи
- Рабочее напряжение: 3.3 В
- Потребляемый ток: до 500 мА
- Контроллер: ESP-WROOM-32, 2-ядерный процессор ESP32-D0WDQ6 на основе 32-битных Xtensa LX6 с низким энергопотреблением
- Тактовая частота: до 240 МГц
- Внутренняя память:
    - 448 Кбайт ПЗУ для загрузки и базовых функций

    - 520 Кбайт статической ОЗУ для данных и инструкций
    - 8 Кбайт быстрой статической ОЗУ для домена питания реального времени (RTC), доступной через главный процессор во время загрузки из режима глубокого сна
    - 8 Кбайт медленной статической ОЗУ для домена питания реального времени (RTC), доступной через сопроцессор в режиме глубокого сна
    - 1 Кбит eFuse, 256 бит заняты системой (MAC-адрес и настройки чипа), 768 бит для пользовательских приложений, включая флеш-шифрование и идентификатор микросхемы (chip-ID)
- Встроенная память; 32 МБит / 4 МБайт ПЗУ, 40МГц
- Диапазон радиочастот: 2.4ГГц-2.5ГГЦ (2412М-2484М)
- WiFi:
    - Клиент, Точка доступа, Клиент+Точка доступа (station, softAP, station+softAP)
    - Протоколы WiFi: 802.11 b/g/n, до 150 Мбит/сек
    - Выходная мощность в режиме 802.11b: +20.5 dBm
    - Поддержка WiFi MAC
    - Технология STBC 2x1
    - Антенна: PCB, разведена на плате в виде дорожки
- Bluetooth:
    - Версия 4.2 BR/EDR и BLE спецификации
    - Мощность передающего сигнала: +12dBm
    - NZIF-приёмник с чувствительностью -97dBm
    - Адаптивная форма скачкообразной перестройки частоты (AFH)
    - Class-1, class-2 и class-3 передатчик без внешнего усилителя мощности
    - Поддержка мульти-соединений в режимах классического BT и BLE
    - Поддержка многоранговых соединений Piconect и Scatternet
    - Поддержка голосовых кодеков CVSD и SBC

- Шифрование: WAPI, WEP, TKIP, AES, SHA-2, RSA, ECC
- Экран: OLED 128x64 точки, контроллер SSD1306
- Выводы общего назначения (вход/выход, GPIO): 32
- Аналоговые входы (АЦП): 12, 12-бит
- Аналоговые выходы (ЦАП): 2, 8-бит
- Максимальный ток на контакт общего назначения: 12 мА, рекомендуемый 6 мА
- Интерфейсы: GPIO, UART, I2C, I2S, SPI, PWM, Touch, Ethernet MAC, SDcard, SDIO, IrDA
- Скорость передачи данных UART: до 5 МБит/сек
- Встроенный переключатель приёма/передачи, согласующий высокочастотный трансформатор, усилитель мощности
- Встроенные блоки: согласования сети, фазовой автоподстройки частоты, управления питанием, блоки регулирования
- Встроенные: датчик Холла, ёмкостный датчик касаний
- Поддержка Arduino, NodeMCU, MicroPhyton, ESP-IDF, PlatformIO
- Поддержка файловых систем SPIFFS, FATFS
- Поддержка Программных наборов разработки (SDK), обновление прошивки по UART/OTA
- Программируемый светодиод вывода GPIO16
- Автономное питание:
    - Контроллер заряда/разряда: TP5400
    - Ток заряда: 500 мА
    - Индикаторы режима заряда
    - Совместимая батарея: литиевая, 18650 (не входит в комплектацию)
    - Батарейный отсек-держатель
- Выключатель управления питанием
- Шаг между контактами: 2.54 мм
- Размеры: 115.0 x 29.3 x 20.0 мм

4.3. Выбор дальномера

Существует множество устройств, которые могут измерять расстояния, некоторые из них измеряют расстояния на основе ультразвука. Однако нам необходима высочайшая точность, для этого понадобится оптический дальномер. Этот тип измерительных приборов основан на лазере. В процессе работы над выпускной квалификационной работы был выбран датчики VL53L0/1X (рисунок 11), принцип работы которого основан на методе Time of Flight.

Рисунок 11 – Дальномер VL53L0/1X

Time of Flight или ToF - это метод измерения расстояний, основанный на оптике, измеряющей время, прошедшее между испусканием светового луча и его приемом. Это могут быть датчики CCD, CMOS, а импульсы могут быть инфракрасными, лазерными и т.д. Система должна быть синхронизирована, чтобы начать измерение времени как раз при срабатывании импульса и остановить счетчик, когда они получат дребезг от датчика (рисунок 12) [10].

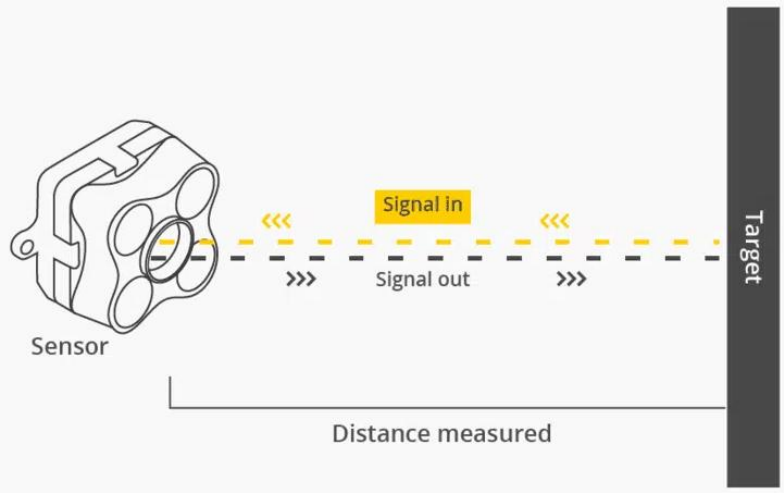


Рисунок 12 – Метод измерения расстояния Time of Flight

Технические характеристики VL53L0/1X-V2 [9]:

- Интерфейс: I2C(TWI)
- Диапазон измерения расстояния: 30 … 4000 мм
- Точность датчика: ± 3%
- Инфракрасный диапазон излучения: 940 нм
- Напряжение питания: 3,3 … 6 В
- Ток потребления: 10 мА
- Пиковый ток не более: 40 мА
- Размеры: 18 x 13 x 2 мм
- Вес: 2 г

Для работы с ESP-WROOM-32 необходимо установить библиотеку VL53L0X library for Arduino [11].

4.4. Взаимодействие между устройством и облачной платформой Rightech IoT Cloud

Взаимодействие облачной платформы Rightech IoT Cloud с ESP-WROOM-32 осуществляется с помощью интерфейса Wi-Fi по протоколу MQTT (рисунок 13). Для авторизации ESP-WROOM-32 в облачной платформе RIC используется логин и пароль, которые задаются пользователем.

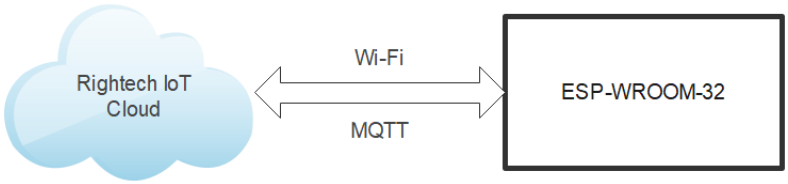


Рисунок 13 – Взаимодействие облачной платформы RIC с ESP--WROOM-32

Протокол MQTT (Message Queue Telemetry Transport) – это легкий, компактный и открытый протокол обмена данными, созданный для передачи информации на удаленных локациях, где объем передаваемых данных невелик и есть ограничения по пропускной способности канала. К основным преимуществам протокола MQTT можно отнести следующие:

- компактные передаваемые сообщения;
- работа в условиях нестабильного соединения;
- легкая интеграция новых устройств;
- поддержка нескольких уровней качества обслуживания.

Протокол MQTT работает на прикладном уровне поверх протокола транспортного уровня TCP и протокола сетевого уровня IP (рисунок 14) [12].

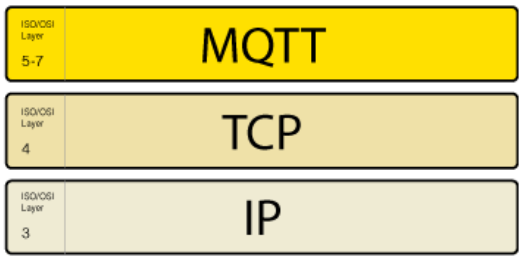


Рисунок 14– Уровни протоколов передачи данных

Обмен сообщениями в протоколе MQTT осуществляется между клиентом («client»), который может быть издателем («publisher») или подписчиком («subscriber») сообщений, и брокером («broker») сообщений. Платформа «RIC» предоставляет доступ к собственному MQTT-брокеру по адресу dev.rightech.io, используя 1883 порт для подключения [8].

Издатель отправляет данные на MQTT-брокер, указывая в сообщении определенную тему, топик («topic»). Подписчики данного топика могут получать разные данные от множества издателей, также один издатель может публиковать сообщения в множество топиков.

Устройства MQTT используют определенные типы сообщений для взаимодействия с брокером, ниже представлены основные:

- «connect» – установить соединение с брокером;
- «disconnect» – разорвать соединение с брокером;
- «publish» – опубликовать данные в топик на брокере;
- «subscribe» – подписаться на топик на брокере;
- «unsubscribe» – отписаться от топика.

Для наглядного описания взаимодействия между издателем, брокером и клиентом была составлена UML-диаграмма последовательности.

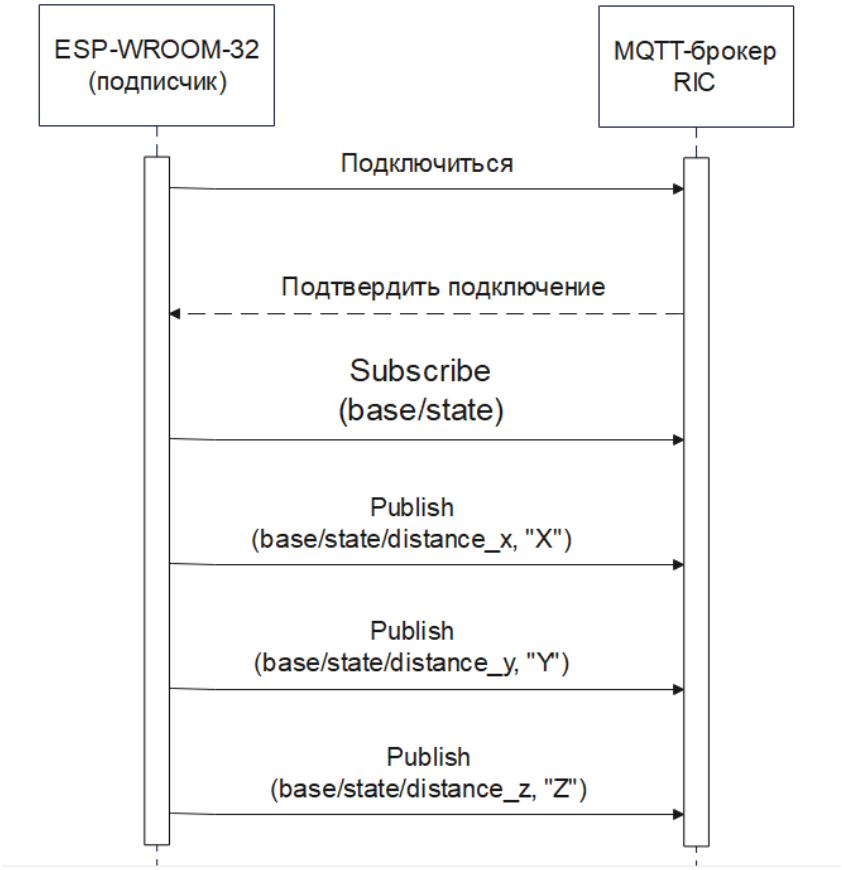


Рисунок 15 – UML-диаграмма последовательности взаимодействия ESP-WROOM-32 с облаком

Прежде всего создается TCP/IP соединение между устройством и облачной платформой. После чего сервер устройства подключается к MQTT-брокеру, получает подтверждение подключения и подписывается на определенный топик брокера.

Для подписки клиента на определенный топик используется следующий формат сообщения: «base/state», где «state» – название топика, в который MQTT-брокер публикует команды.

Для защиты информации, передаваемой между устройствами в RIC в облаке, применяется криптографический протокол TLS 1.2. Этот протокол позволяет аутентифицировать устройства в системе с использованием X509 сертификатов или логина и пароля. В разрабатываемой системе аутентификация устройств происходит через логин и пароль, где ID устройства выступает в качестве логина, а пароль задается пользователем. Использование шифрования позволяет минимизировать риски кражи данных третьими лицами, поскольку доступ к логину и паролю каждого устройства ограничен.

5. Рабочее проектирование

Перед тем, как приступать к написанию программной части работы, надо разработать схему подключать устройства контроля положения штабелера к облачной платформе RIC и разработать алгоритмы работы основных функции.

Для программной части устройства основными функциями являются:

Примечание [BSA4]: а интерфейс?

- снятие данных с датчиков;
- отправка данных на облачную платформу.

Программная часть облачной системы состоит из одной функции обработки полученных данных от устройства контроля положения штабелера.

5.1. Принципиальная схема подключения ESP-WROOM-32 к дальномерам

Принципиальная схема подключения Wemos TTGO WiFi/Bluetooth BLE с OLED-экраном 0.96 дюйма на контроллере ESP-WROOM-32 ESP 32 к 3-м дальномерам VL53L1/0X-V2 представлена на рисунке 16 [13].

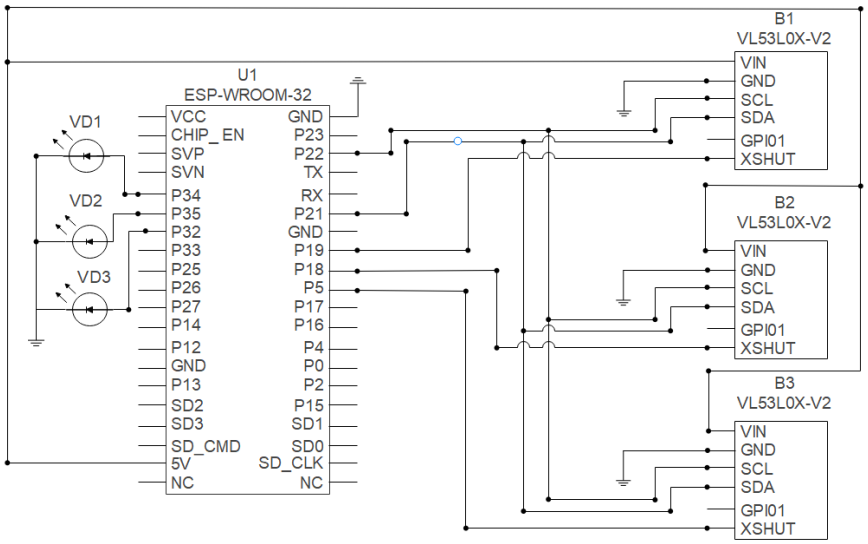


Рисунок 16 – Принципиальная схема подключения контроллера к дальномерам

В данной принципиальной схеме также используются светодиоды для индикации корректной работы дальномеров. При ошибке подключения дальномера к ESP 32 загорается соответствующий светодиод. В таблице 1 представлен перечень элементов принципиальной схемы подключения контроллера к дальномерам.

Таблица 1 – Перечень элементов принципиальной схемы

Поз. Обозначение	Наименование	Кол.-во	Примечание
U1	Контроллер ESP-WROOM-32	1	
B1, B2, B3	Дальномер VL53L1/0X-V2	3	
VD1, VD2, VD3	Индикаторный светодиод	3	

5.2. Разработка алгоритма работы устройства контроля положения

Алгоритм работы устройства контроля положения штабелера представлен на рисунке 17 [14]. В данном алгоритме в первую очередь выполняется проверка корректности подключения дальномеров и установки им новых адресов, т.к. взаимодействие дальномеров с контроллером ESP-WROOM-32 происходит по двунаправленному последовательному интерфейсу I2C/IIC, состоящий из линии данных SDA и тактирования SCL. Если происходит ошибка подключения одного из дальномеров, то высвечивается ошибка и загорается соответствующий индикаторный светодиод.

После проверки корректности настройки датчиков, происходит подключение к сети Wi-Fi и MQTT-брокеру RIC, затем – подписка на топик. После чего запускается бесконечный цикл (loop), где контроллер считывает данные с дальномеров и отправляет их на облачную платформу с заданной периодичностью.

Для работы с Wi-Fi и MQTT используется библиотека MQTT and Wifi handling for ESP8266 and ESP32 [15].

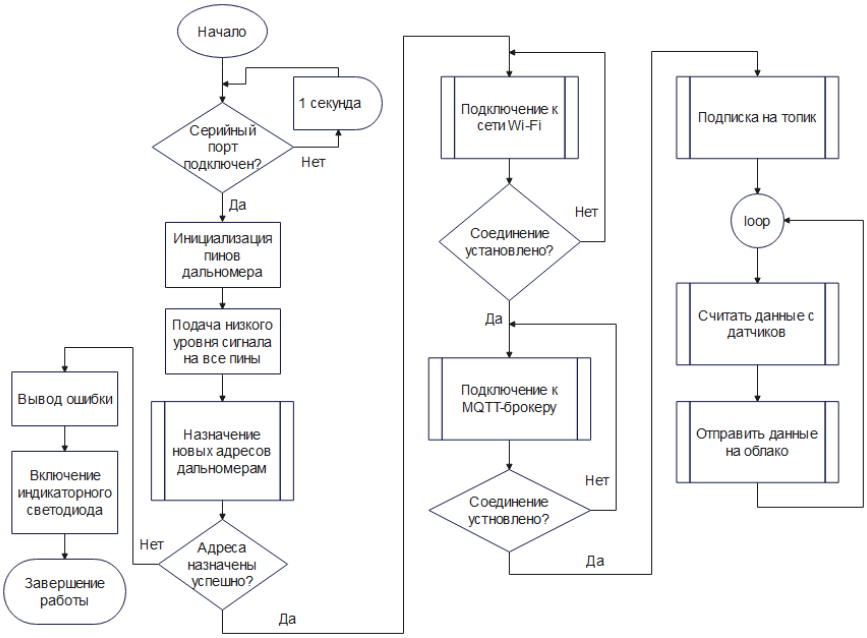


Рисунок 17 – Алгоритм работы основной программы устройства контроля положения штабелера

Также была составлена блок-схема алгоритма работы функции “Считать данные с датчиков” (рисунок 18) [14].

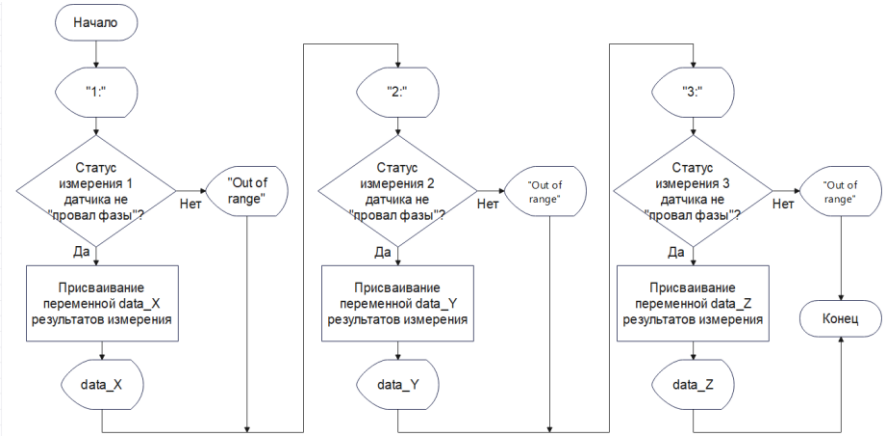


Рисунок 18 – Алгоритм работы функции “Считать данные с датчиков”

Дальномер VL53L1/0X-V2 имеет 7 статусов измерений. В данном алгоритме нам необходим “статус измерений 1”, что означает “провал фазы” [11]. Если провала фазы нет, то значит измеряемое расстояние находится в доступном для датчика диапазоне и можно присваивать переменной data_X результаты измерений. Если же измеряемое расстояние больше диапазона датчика, то выдается сообщение “Out of range” (За пределами измерения). То же самое для остальных двух дальномеров.

Алгоритм работы функции “Отправить данные на облако” представлен на рисунке 19 [14].

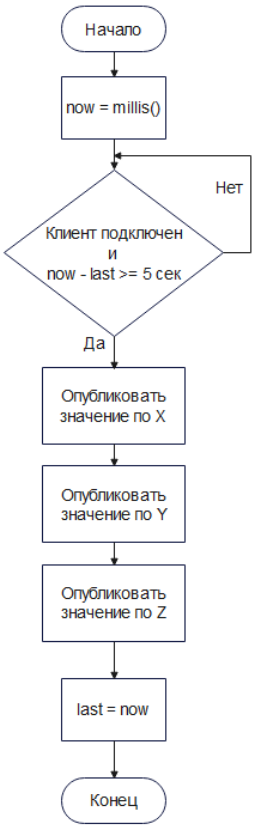


Рисунок 18 – Алгоритм работы функции “Отправить данные на облако

”В данном алгоритме используются две переменные last и now, которые обозначают последний и новый моменты отправки данных на облако. Для начала переменной now присваивают значение функции millis(), которая возвращает количество миллисекунд, которое прошло с начала запуска программы.

Далее происходят проверки подключения клиента к MQTT-брокеру и условия, что между отправкой предыдущего сообщения и настоящего момента прошло более 5 секунд (опционально, данные можно отправлять и с большей задержкой), после чего на топик отправляются значение переменных data_X, data_Y, data_Z, и переменной last присваивается значение now.

Полная программа работы устройства контроля положения штабелера представлена в Приложении А.

5.3. Настройка облачной платформы Rightech IoT Cloud

Чтобы подключить устройство к облачной платформе RIC. Нужно пройти несколько этапов:

Примечание [BSA5]: Нужна блок-схема процесса настройки (сюда и в презентацию)

- Создать MQTT-модель объекта и настроить аргументы (рисунок 19);

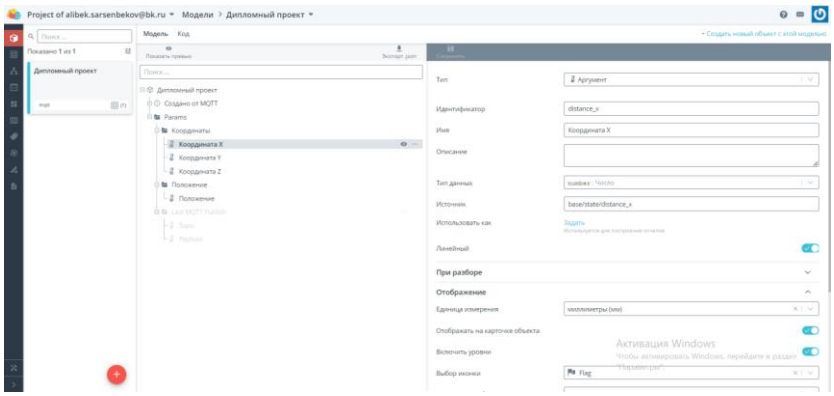


Рисунок 19 – Окно создания MQTT-модели

В данной работе используется четыре аргумента. Первый аргумент – координата по оси X, значение которой приходит непосредственно от устройства на облачную платформу. Второй аргумент – координата Y, третий – Z. Четвертый аргумент – текст, который отображает положение штабелера, если он находится в заранее заданных точках (ячейки, робокар, нулевое положение, конвейер).

- Создать объект по созданной модели, настроить идентификатор и опционально MQTT-логин, MQTT-пароль (рисунок 20);

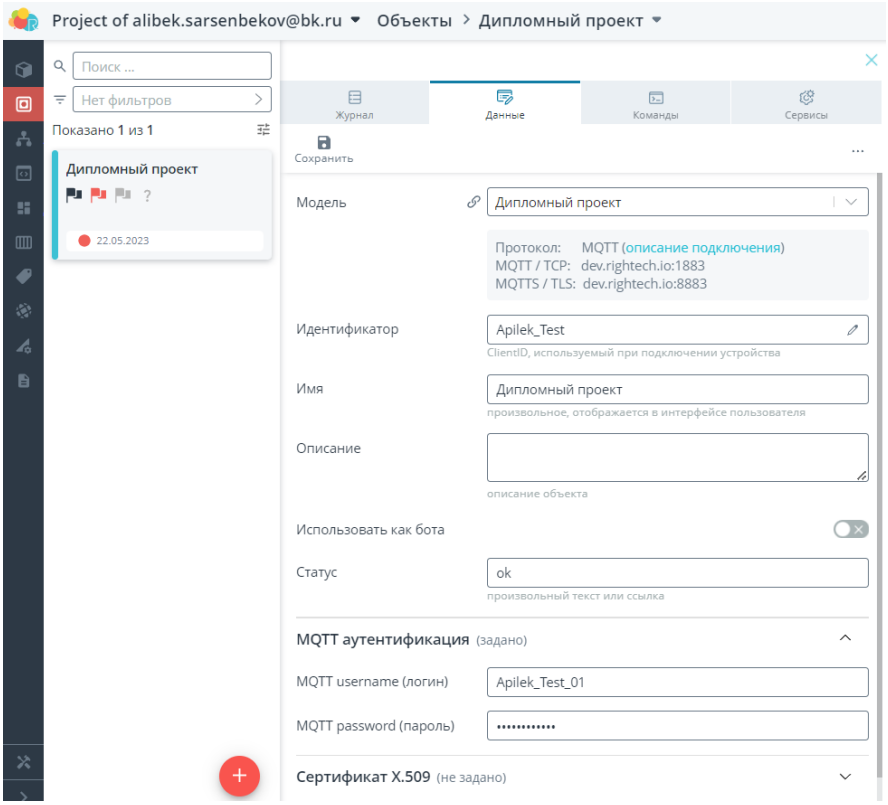


Рисунок 20 – Окно создания и настройки нового объекта

- И последний этап – прошить микроконтроллер ESP-WROOM-32, указав в программе идентификатор, MQTT-логин и MQTT-пароль.

После получения данных облачной платформой от устройства, необходимо их проанализировать и определить положение штабелера. В облачной платформе Rightech IoT Cloud предусмотрена возможность оперировать данными с помощью “обработчика”. Обработчик позволяет обрабатывать входной пакет данных с целью его коррекции или дополнения. Для создания обработчика необходимо написать исходный код на языке программирования JavaScript версии ES6.

Окно обработчика представлено на рисунке 21 и представляет из себя своего рода компилятор.

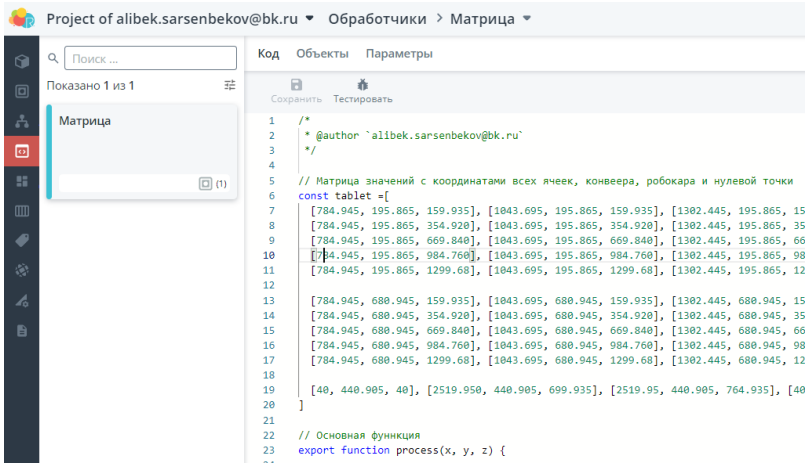


Рисунок 21 – Окно обработчика

Исходный код обработчика данных представлена в Приложении Б. Для обработки данных и вывода положения штабелера, необходимо определить координаты всех ячеек стеллажа склада, координаты конвейера и робокара. В разрабатываемой облачной системе в качестве точки отсчета принимается крайнее правое положение штабелера, при котором нажаты концевые выключатели на каждой из его осей. Координаты ячеек стеллажа представлены в Приложении В.

Чтобы обработчик понимал, откуда брать значения входных и выходных их нужно настроить в отдельной вкладке, которая представлена на рисунке 22.

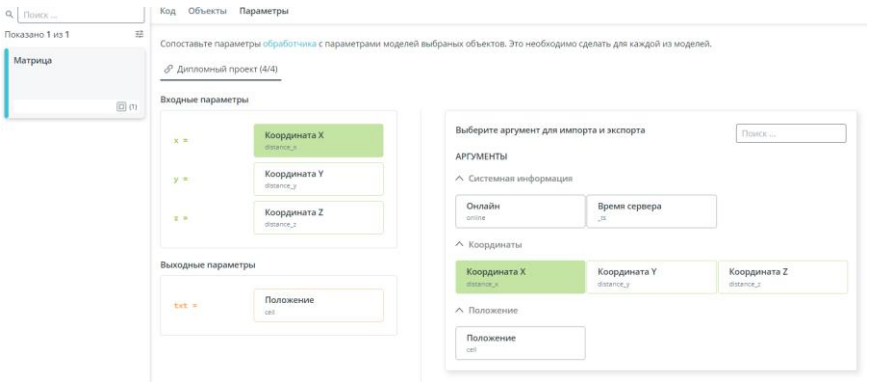


Рисунок 22 – Вкладка с настройками выходных и выходных параметров

Для проверки работоспособности программы до подключения устройства в обработчик встроен компилятор, который помогает протестировать программу (рисунок 23).

Рисунок 23 – Окно тестирования

6. Апробирование

Устройство представляет из себя плату Wemos TTGO WiFi/Bluetooth BLE с OLED-экраном 0.96 дюйма на контроллере ESP-WROOM-32 ESP32 с подключенными к нему тремя дальномерами VL53L1/0X-V2. Датчики устройства должны располагаться на штабелере как на рисунке 24.

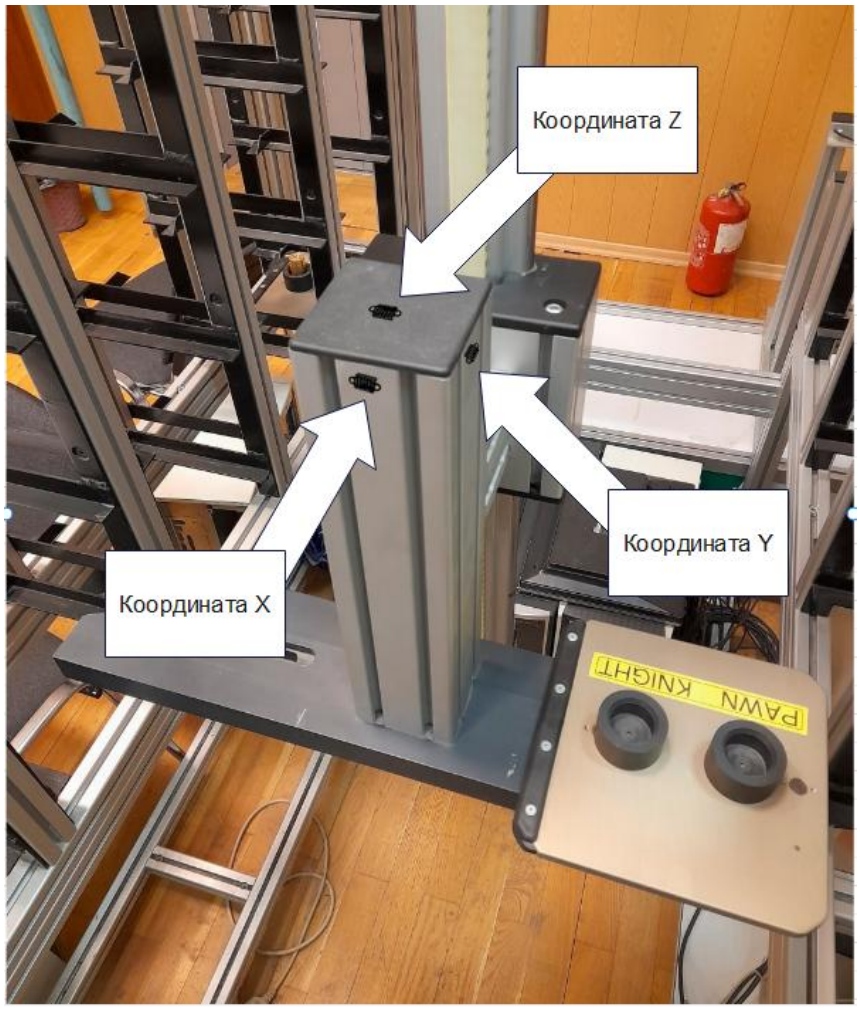


Рисунок 24 – Штабелер с расположением датчиков

Микроконтроллер подключен к беспроводной точке доступа Wi-Fi с выходом в интернет. После загрузки кода программы в микроконтроллер, или после включения микроконтроллера в окне вывода данных последовательного порта (Serial Monitor) выводятся данные о результатах измерения расстояния (рисунок 25).

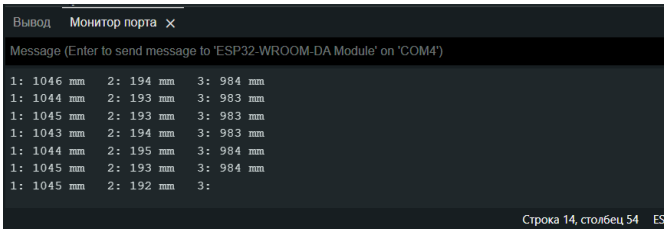


Рисунок 25 – Последовательный монитор порта Arduino с результатами измерений.

В этот же момент отправляются результаты измерений на облачную платформу через определенное количество времени (рисунок 26). Платформа идентифицирует устройство по уникальному ID клиента (ClientID).

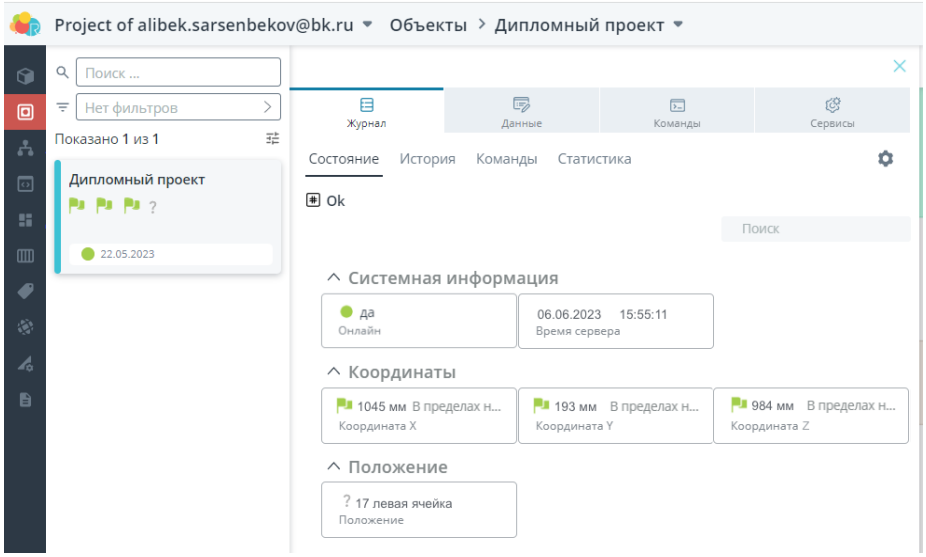


Рисунок 26 – Окно с журналом состояний (устройство подключено и отправляет данные)

В ходе выпускной квалификационной работы также был разработан интерфейс мобильного приложения, который показывает положение штабелера (координаты и само положение). Интерфейс приложения показан на рисунке 27.

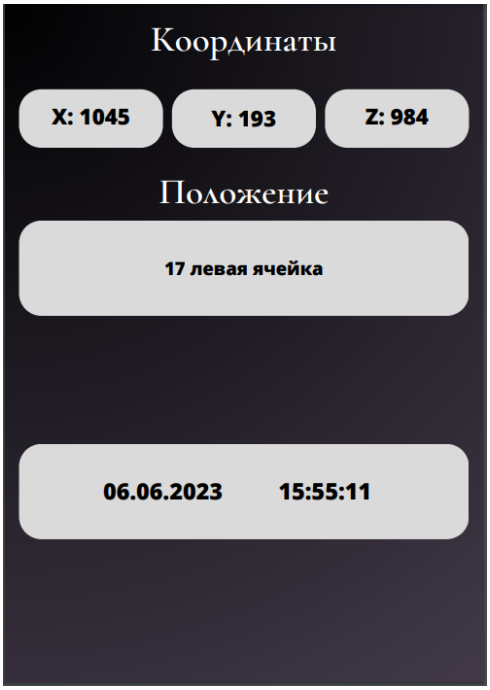


Рисунок 27 – Интерфейс приложения

Как видно из полученных данных апробирования, устройство контроля положения штабелера автоматизированного склада успешно отправило данные на облачную платформу, которая в свою очередь правильно определила положение штабелера.

ЗАКЛЮЧЕНИЕ

В ходе работы над выпускной квалификационной работой были получены следующие результаты:

1. Промышленные предприятия в настоящее время внедряют кибер-физические системы и технологии интернета вещей в производственный процесс, что приводит к замене традиционных систем автоматизации управления технологическими процессами на облачные платформы. В таких системах планирование ресурсов предприятия, контроль и управление процессами осуществляются с помощью облачных технологий..
2. Анализ существующей складской системы показал ряд недостатков, одной из которых является отсутствие системы контроля положения штабелера;
3. ~~Была~~ разработана функциональная модель системы контроля положения штабелера;
4. ~~Была~~ выбрана облачная платформа Rightech IoT Cloud, ~~как самая лучшая для работы в рамках выпускной квалификационной работы~~;
5. Для устройства контроля положения штабелера ~~были~~ выбраны микроконтроллер Wemos TTGO WiFi/Bluetooth BLE с OLED-экраном 0.96 дюйма на контроллере ESP-WROOM-32 ESP32 и дальномеры VL53L1/0X;
6. ~~Были~~ разработаны алгоритмы работы основных функций устройства контроля положения штабелера: функция чтения данных с датчиков и функции отправки данных на облачную платформу;
7. ~~Был~~ разработан исходный код устройства контроля положения штабелера, который измеряет расстояние и отправляет данные на облачную платформу, на языке C++;
8. ~~Была~~ разработана принципиальная электрическая схема подключения микроконтроллера ESP-WROOM-32 к дальномерам VL53L1/0X;

9. ~~Был~~ разработан исходный код обработчика облачной платформы Rightech IoT Cloud для анализа полученных от устройства данных и определения положения штабелера на языке JavaScript версии ES6;
10. ~~Было~~ собран~~о~~ прототип ~~устройство~~ устройства контроля положения штабелера автоматизированного склада;
11. ~~Был~~ разработан интерфейс мобильного приложения, который показывает актуальные координаты штабелера и его положение, если тот не находится в промежуточных позициях.
12. Проведено ~~Апробирование~~ тестирование, показав~~ло~~шее работоспособность системы, которая в дальнейшем может быть включена в общую ГПС Denford и взаимодействовать с другими модулями ГПС.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Lee E. A., Seshia S.A. Introduction to Embedded Systems. – A Cyber-Physical Systems Approach. – Berkley: UC Berkley, 2011. – 519 p.
2. Соколов Д. Индустрия 4.0: Big Data, цифровизация и рост экономики. [Электронный ресурс] // Habr [сайт]. [2020]. URL: https://habr.com/ru/post/507822/ (дата обращения 12.10.2022)
3. Овсянников М. В., Буханов С. А., Подкопаев С. А. Организация индустриального интернета на основе SAP HANA Cloud Platform: Учебн. пособие. – М.: издательство МГТУ им. Н. Э. Баумана, 2019. – 195 с.
4. Buyya R., Dastjerdi A.V. Internet of Things Principles and Paradigms. – Singapore: Elsevier, 2016. – 354 p.
5. Р 50.1.028-2001 Информационные технологии поддержки жизненного цикла продукции. Методология функционального моделирования
6. ГОСТ 34.602-89 Техническое задание на создание автоматизированной системы.
7. Облачные платформы [Электронный ресурс] // iot.ru [сайт] URL: https://iot.ru/wiki/oblachnye-platformy (дата обращения 13.10.2022)
8. Введение для разработчиков [Электронный ресурс] // Rightech [сайт] URL: https://rightech.io/developers/introductions/ (дата обращения 10.11.2022)
9. Модуль Wemos TTGO WiFi/Bluetooth BLE с OLED-экраном 0.96 дюйма на контроллере ESP-WROOM-32 ESP32 [Электронный ресурс] // Compact tool [сайт] URL: https://compacttool.ru/wemos-ttgo-wifibluetooth-ble-s-oled-ekranom-096-dyujma-na-kontrollere-esp-wroom-32-esp32 (дата обращения: 15.11.2022)

10. VL53L0X: высокоточный лазерный датчик расстояния [Электронный ресурс] // hwlibre.com [сайт] URL: https://www.hwlibre.com/ru/vl53l0x/ (дата обращения: 15.11.2022)
11. VL53L0X library for Arduino [Электронный ресурс] // GitHub [сайт] URL: https://github.com/pololu/vl53l0x-arduino (дата обращения: 15.11.2022)
12. Puler T. Hands-On Internet of Things with MQTT. – Birmingham: Packt, 2019. – 350 p.
13. ГОСТ 2.702-2011. Единая система конструкторской документации. Схемы. Виды и типы. Общие требования к выполнению.
14. ГОСТ 19.701-90. Единая система программной документации. Схемы алгоритмов, программ, данных и систем
15. MQTT and Wifi handling for ESP8266 and ESP32 [Электронный ресурс] // GitHub [сайт] URL: https://github.com/plapointe6/EspMQTTClient (дата обращения 15.11.2022)

Приложение А

```
// Подключаем библиотеки
#include "Arduino.h"
#include "EspMQTTClient.h"
#include "Adafruit_VL53L0X.h"

// Определяем время ожидания отправки сигнала (5 секунд)
#define PUB_DELAY (10 * 1000)

// Инициализируем адреса для 3-х дальномеров
#define LOX1_ADDRESS 0x30
#define LOX2_ADDRESS 0x31
#define LOX2_ADDRESS 0x32

// Инициализируем пины для 3-х дальномеров
#define SHT_LOX1 18
#define SHT_LOX2 19
#define SHT_LOX3 19

// Инициализируем пины для индикаторов
#define LOX1_LED 34
#define LOX2_LED 35
#define LOX3_LED 32

// Создаем объекты 3-х дальномеров
Adafruit_VL53L0X lox1 = Adafruit_VL53L0X();
Adafruit_VL53L0X lox2 = Adafruit_VL53L0X();
Adafruit_VL53L0X lox3 = Adafruit_VL53L0X();

// Создаем переменные для измерений
VL53L0X_RangingMeasurementData_t measure_X;
VL53L0X_RangingMeasurementData_t measure_Y;
VL53L0X_RangingMeasurementData_t measure_Z;

// Создаём переменные для вывода
int data_X, data_Y, data_Z;
```

```
void setID() {

  // Сброс 3-х дальномеров
  digitalWrite(SHT_LOX1, LOW);
  digitalWrite(SHT_LOX2, LOW);
  digitalWrite(SHT_LOX3, LOW);
  delay(10);
  // Отмена сброса
  digitalWrite(SHT_LOX1, HIGH);
  digitalWrite(SHT_LOX2, HIGH);
  digitalWrite(SHT_LOX3, HIGH);
  delay(10);

  // Активация LOX1 и перезагрузка LOX2, LOX3
  digitalWrite(SHT_LOX1, HIGH);
  digitalWrite(SHT_LOX2, LOW);

  // Инициализация LOX1
  if(!lox1.begin(LOX1_ADDRESS)) {
    Serial.println(F("Failed to boot first VL53L0X"));
    digitalWrite(LOX1_LED, HIGH);
    while(1);
  }
  delay(10);

  // Активация LOX2
  digitalWrite(SHT_LOX2, HIGH);
  delay(10);

  // Инициализация LOX2
  if(!lox2.begin(LOX2_ADDRESS)) {
    Serial.println(F("Failed to boot second VL53L0X"));
    digitalWrite(LOX2_LED, HIGH);
    while(1);
  }

  digitalWrite(SHT_LOX3, HIGH);
  delay(10);
```

```
  // Инициализация LOX3
  if(!lox2.begin(LOX3_ADDRESS)) {
    Serial.println(F("Failed to boot third VL53L0X"));
    digitalWrite(LOX3_LED, HIGH);
    while(1);
  }
}

void read_trio_sensors() {

  // переключить на 'true', чтобы получить распечатку
отладочных данных
  lox1.rangingTest(&measure_X, false);
  lox2.rangingTest(&measure_Y, false);
  lox3.rangingTest(&measure_Z, false);

  // Вывести результат измерения 1 дальномера
  Serial.print("1: ");
  if(measure_X.RangeStatus != 4) {     // если не за
пределами измерения
    data_X = measure_X.RangeMilliMeter;
    Serial.print(data_X);
    Serial.print("mm");
  } else {
    Serial.print("Out of range");
  }

  Serial.print(" ");

  // Вывести результат измерения 2 дальномера
  Serial.print("2: ");
  if(measure_Y.RangeStatus != 4) {
    data_Y = measure_Y.RangeMilliMeter;
    Serial.print(data_Y);
    Serial.print("mm");
  } else {
    Serial.print("Out of range");
  }
```

```
  Serial.print(" ");

  // Вывести результат измерения 3 дальномера
  Serial.print("3: ");
  if(measure_Z.RangeStatus != 4) {
    data_Z = measure_Z.RangeMilliMeter;
    Serial.print(data_Z);
    Serial.print("mm");
  } else {
    Serial.print("Out of range");
  }

  Serial.print("\n");
}

// Регистрация устройства
EspMQTTClient client(
  "Apilek",
  "A12345671",
  "dev.rightech.io",
  "Apilek_Test"
);

// Проверка подключения контроллера и настройка пинов
void setup()
{
  Serial.begin(115200);
  while (! Serial)
  {
    delay(1);
  }
  pinMode(SHT_LOX1, OUTPUT);
  pinMode(SHT_LOX2, OUTPUT);
  pinMode(SHT_LOX3, OUTPUT);

  pinMode(LOX1_LED, OUTPUT);
  pinMode(LOX2_LED, OUTPUT);
  pinMode(LOX3_LED, OUTPUT);
```

```
  Serial.println("Shutdown pins inited...");

  digitalWrite(SHT_LOX1, LOW);
  digitalWrite(SHT_LOX2, LOW);
  digitalWrite(SHT_LOX3, LOW);

  digitalWrite(LOX1_LED, LOW);
  digitalWrite(LOX2_LED, LOW);
  digitalWrite(LOX3_LED, LOW);

  Serial.println("All three in reset mode...(pins are
low)");

  Serial.println("Starting...");
  setID();
}

// Установление соединения с облаком
void onConnectionEstablished() {
  client.subscribe("base/state", [] (const String
&payload)  {
    Serial.println(payload);
  });
}

// Инициализация переменной времени последнего измерения
long last = 0;

// Функция отправки результатов измерения
void publishDistance()
{
  // Считывание результатов датчиков
  read_trio_sensors();
  // Инициализация переменной времени нового измерения
  long now = millis();
    // Проверка времени ожидания
  if (client.isConnected() && (now - last > PUB_DELAY))
 {
  // Отправка результатов
```

```
  client.publish("base/state/distance_x",
String(data_X));
  client.publish("base/state/distance_y",
String(data_Y));
  client.publish("base/state/distance_z",
String(data_Z));
  // Обновление переменной времени
  last = now;
  }
}

// Бесконечный цикл
void loop()
{
  client.loop();
  publishDistance();
}
```

Приложение Б

```
// Матрица значений с координатами всех ячеек, конвеера,
робокара и нулевой точки
const tablet =[
  [784.945, 195.865, 159.935], [1043.695, 195.865,
159.935], [1302.445, 195.865, 159.935], [1561.195,
195.865, 159.935], [1819.945, 195.865, 159.935],
  [784.945, 195.865, 354.920], [1043.695, 195.865,
354.920], [1302.445, 195.865, 354.920], [1561.195,
195.865, 354.920], [1819.945, 195.865, 354.920],
  [784.945, 195.865, 669.840], [1043.695, 195.865,
669.840], [1302.445, 195.865, 669.840], [1561.195,
195.865, 669.840], [1819.945, 195.865, 669.840],
  [784.945, 195.865, 984.760], [1043.695, 195.865,
984.760], [1302.445, 195.865, 984.760], [1561.195,
195.865, 984.760], [1819.945, 195.865, 984.760],
  [784.945, 195.865, 1299.68], [1043.695, 195.865,
1299.68], [1302.445, 195.865, 1299.68], [1561.195,
195.865, 1299.68], [1819.945, 195.865, 1299.68],

  [784.945, 680.945, 159.935], [1043.695, 680.945,
159.935], [1302.445, 680.945, 159.935], [1561.195,
680.945, 159.935], [1819.945, 680.945, 159.935],
  [784.945, 680.945, 354.920], [1043.695, 680.945,
354.920], [1302.445, 680.945, 354.920], [1561.195,
680.945, 354.920], [1819.945, 680.945, 354.920],
  [784.945, 680.945, 669.840], [1043.695, 680.945,
669.840], [1302.445, 680.945, 669.840], [1561.195,
680.945, 669.840], [1819.945, 680.945, 669.840],
  [784.945, 680.945, 984.760], [1043.695, 680.945,
984.760], [1302.445, 680.945, 984.760], [1561.195,
680.945, 984.760], [1819.945, 680.945, 984.760],
```

```
  [784.945, 680.945, 1299.68], [1043.695, 680.945,
1299.68], [1302.445, 680.945, 1299.68], [1561.195,
680.945, 1299.68], [1819.945, 680.945, 1299.68],

  [40, 440.905, 40], [2519.950, 440.905, 699.935],
[2519.95, 440.905, 764.935], [40, 440.905, 1299.680]
]

// Основная фуннкция
export function process(x, y, z) {

// Переменная индекса положения в матрице значений
let cell = 0;
let txt = "";

// Максмальные значения координат по каждой из оси
const max_valueX = 2485;
const max_valueY = 685;
const max_valueZ = 1265;

// Отклонение значений
const deviation = 5;

// Проверка максимальных значение по каждой из оси
if (x > max_valueX || y > max_valueY || z > max_valueZ)
{
  if (x > max_valueX)
  {
    let k = x - max_valueX;
    txt = "X > max X на " + k + " мм; ";
  }
  if (y > max_valueY)
  {
    let k = y - max_valueY;
```

```
    txt = txt + "Y max > на " + k + " мм; ";
  }
  if (z > max_valueZ)
  {
    let k = z - max_valueZ;
    txt = txt + "Z max > на " + k + " мм";
  }
   return { txt };
}
else
{
  // Цикл, который сравнивает значение координат с
датчиков с матрицей значений с учетом отклонения
  for (let i = 0; i < 55; i++) {
      if (Math.abs(tablet[i][0] - x) < deviation &&
Math.abs(tablet[i][1] - y) < deviation &&
Math.abs(tablet[i][2] - z) < deviation)
        {
          cell = i;
          i = 54;
        }
 }

  // Вывод результата в зависимости от индекса ячейки в
матрице значений
  if (cell < 25)
    {
      cell = cell + 1;
      txt = cell.toString()  + " левая ячейка ";
      return { txt };
    }
  else if (cell < 50)
    {
      cell = cell - 24;
```

```
      txt = cell.toString() + " правая ячейка";
      return { txt };
    }
  else if (cell == 50)
    {
      txt = "Нулевая точка";
      return {txt};
    }
  else if (cell == 51)
    {
      txt = "Взять с конвейера";
      return {txt};
    }
  else if (cell == 52)
    {
      txt = "Поставить на конвейер";
      return {txt};
    }
  else if (cell == 53)
    {
      txt = "Робокар";
      return {txt};
    }
}

}
```

Приложение В

Координаты ячеек стеллажа склада

Левый стеллаж			
№ ячейки	X	Y	Z
1	784,945	195,865	159,935
2	1043,695	195,865	159,935
3	1302,445	195,865	159,935
4	1561,195	195,865	159,935
5	1819,945	195,865	159,935
6	784,945	195,865	354,92
7	1043,695	195,865	354,92
8	1302,445	195,865	354,92
9	1561,195	195,865	354,92
10	1819,945	195,865	354,92
11	784,945	195,865	669,84
12	1043,695	195,865	669,84
13	1302,445	195,865	669,84
14	1561,195	195,865	669,84
15	1819,945	195,865	669,84
16	784,945	195,865	984,76
17	1043,695	195,865	984,76
18	1302,445	195,865	984,76
19	1561,195	195,865	984,76

20	1819,945	195,865	984,76
21	784,945	195,865	1299,68
22	1043,695	195,865	1299,68
23	1302,445	195,865	1299,68
24	1561,195	195,865	1299,68
25	1819,945	195,865	1299,68

Правый стеллаж			
№ ячейки	X	Y	Z
1	784,945	680,945	159,935
2	1043,695	680,945	159,935
3	1302,445	680,945	159,935
4	1561,195	680,945	159,935
5	1819,945	680,945	159,935
6	784,945	680,945	354,92
7	1043,695	680,945	354,92
8	1302,445	680,945	354,92
9	1561,195	680,945	354,92
10	1819,945	680,945	354,92
11	784,945	680,945	669,84
12	1043,695	680,945	669,84
13	1302,445	680,945	669,84
14	1561,195	680,945	669,84
15	1819,945	680,945	669,84
16	784,945	680,945	984,76

17	1043,695	680,945	984,76
18	1302,445	680,945	984,76
19	1561,195	680,945	984,76
20	1819,945	680,945	984,76
21	784,945	680,945	1299,68
22	1043,695	680,945	1299,68
23	1302,445	680,945	1299,68
24	1561,195	680,945	1299,68
25	1819,945	680,945	1299,68

Нулевое положение	40	440,905	40
Взять с конвейера	2519,95	440,905	699,935
Поставить на конвейер	2519,95	440,905	764,935
Робокар	40	440,905	1299,68

Приложение Г

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

Выпускная квалификационная работа на тему:

Облачная система контроля положения штабелера автоматизированного склада

Выполнил студент группы РК9-82Б:
Сарсенбеков А.Ж.

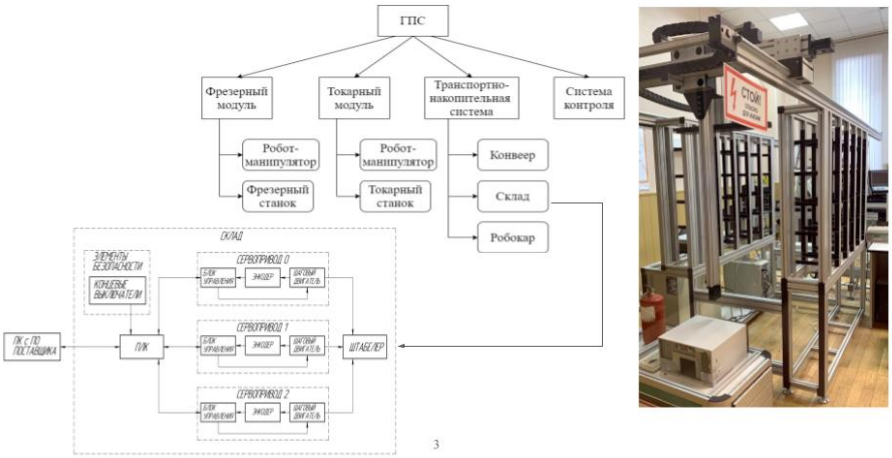
Научный руководитель:
Инженер, старший преподаватель
Буханов С.А.

1

Анализ предметной области

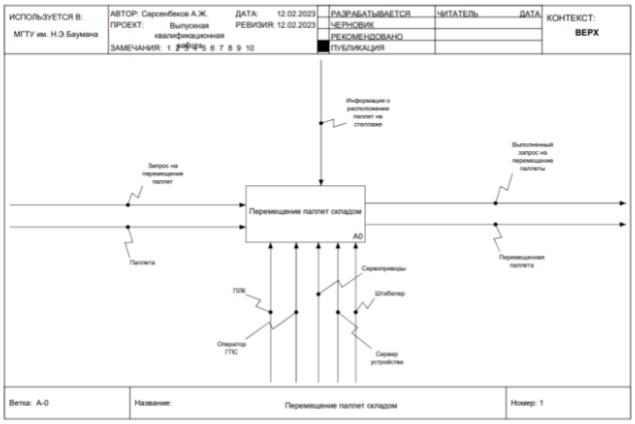
2

Анализ существующей системы



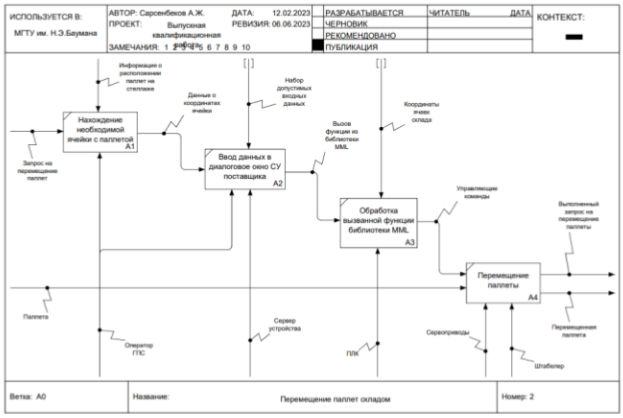
3

Функциональная модель перемещения паллеты складом «as is» (блок А-0)



4

Функциональная модель перемещения паллеты складом «as is» (блок A0)



5

Цель и задачи

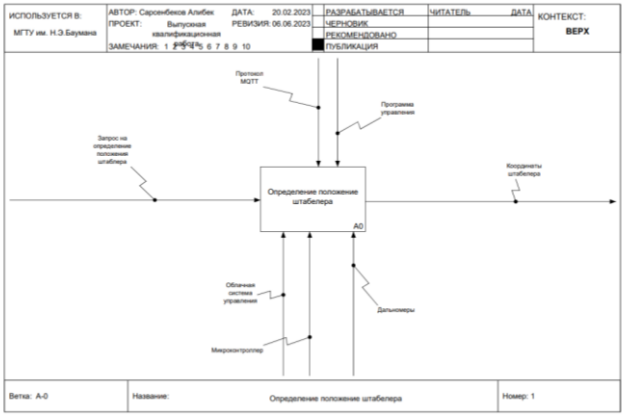
Цель работы: разработать систему контроля положения штабелера складской системы в составе гибкой производственной системы, которая взаимодействует с облачной системой управления.

В ходе работы над проектом были поставлены следующие **задачи**:

1) проанализировать существующую систему управления складской системой и выявить ее недостатки;
2) на основе анализа полученных данных сформулировать требования, предъявляемые к разрабатываемой системе;
3) разработать функциональную модель работы системы контроля положения штабелера;
4) подобрать облачную платформу для обработки результатов измерений;
5) разработать устройство контроля положения штабелера;
6) разработать алгоритмы основных функций устройства;
7) разработать исходный код программной части облачной системы контроля положения штабелера;
8) разработать интерфейс приложения, который будет показывать актуальное положение штабелера.

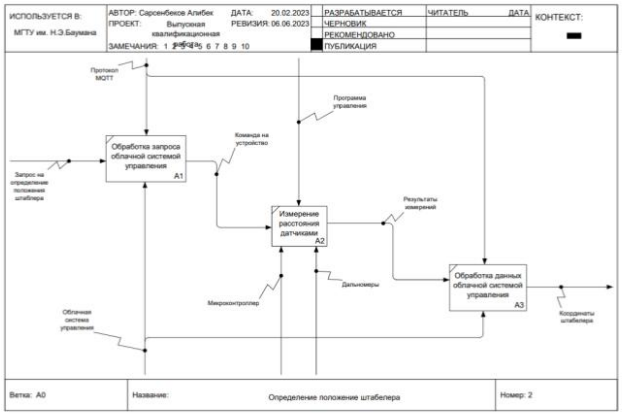
6

Функциональная модель системы контроля положения штабелера (блок А-0)



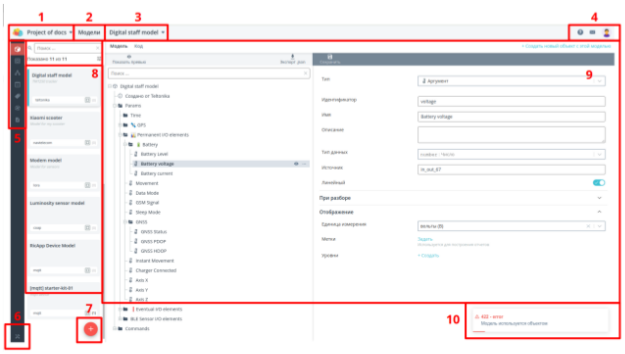
7

Функциональная модель системы контроля положения штабелера (блок А0)



8

Выбор облачной платформы



9

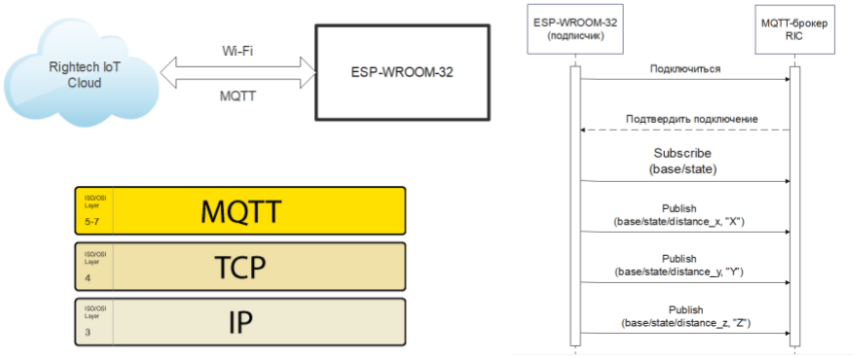
Выбор микроконтроллера и дальномеров

Wemos TTGO WiFi/Bluetooth BLE с OLED-экраном 0.96 дюйма на контроллере ESP-WROOM-32 ESP32

Дальномер VL53L0/1X

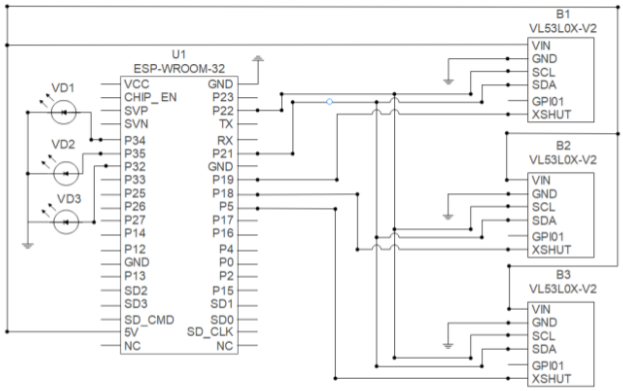
10

Взаимодействие устройства с облачной платформой



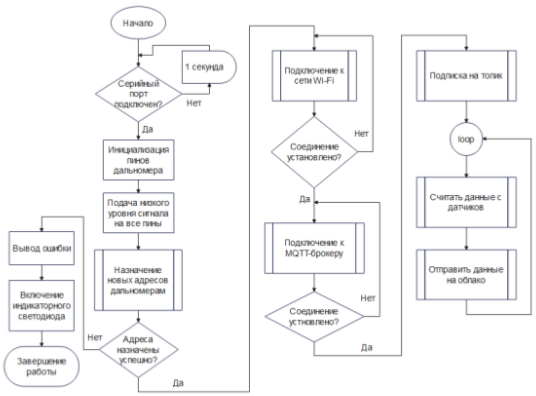
11

Принципиальная схема подключения микроконтроллера к дальномерам



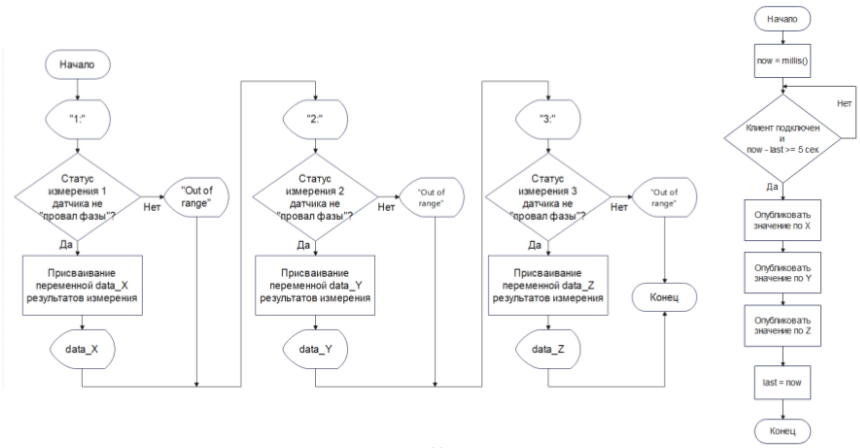
12

Алгоритм работы основной программы устройства



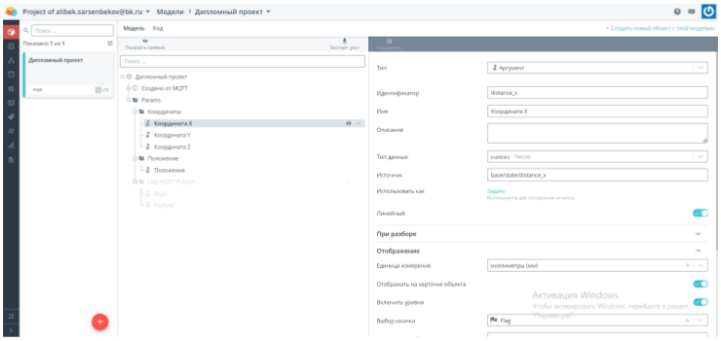
13

Алгоритмы работы основных функций программы устройства



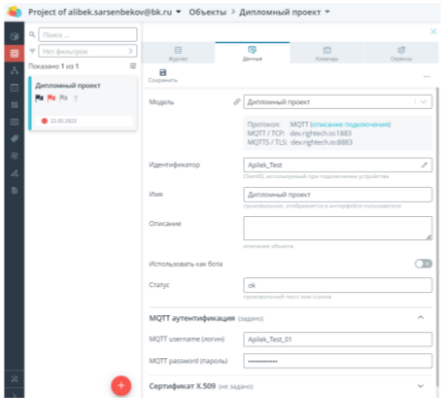
14

Создание модели



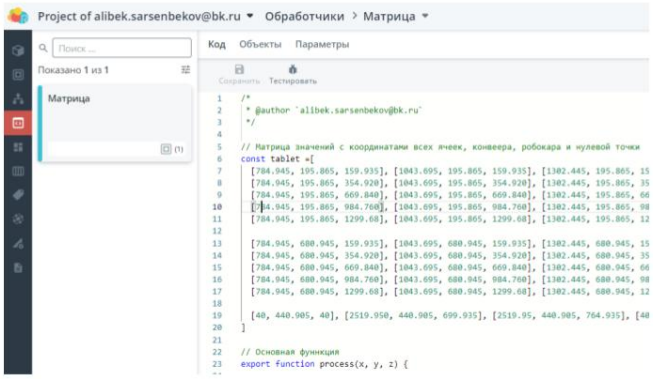
15

Создание объекта



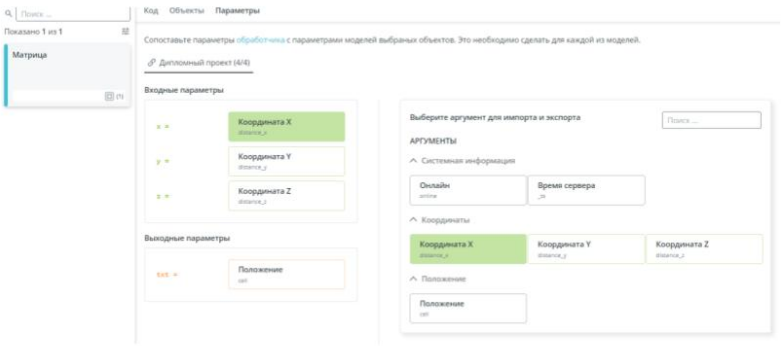
16

Создание обработчика



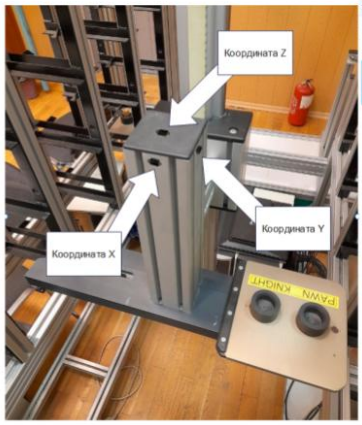
17

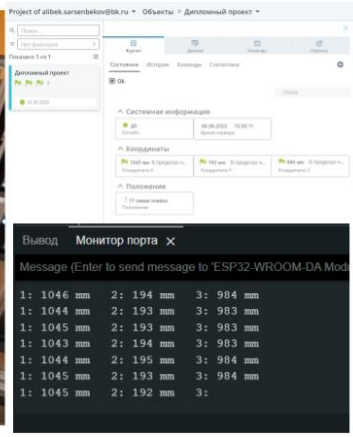
Настройка входных и выходных параметров обработчика



18

Апробирование





19

Заключение

1. Промышленные предприятия в настоящее время внедряют кибер-физические системы и технологии интернета вещей в производственный процесс, что приводит к замене традиционных систем автоматизации управления технологическими процессами на облачные платформы. В таких системах планирование ресурсов предприятия, контроль и управление процессами осуществляются с помощью облачных технологий..
2. Анализ существующей складской системы показал ряд недостатков, одной из которых является отсутствие системы контроля положения штабелера;
3. Была разработана функциональная модель системы контроля положения штабелера;
4. Была выбрана облачная платформа Rightech IoT Cloud, как самая лучшая для работы в рамках выпускной квалификационной работы;
5. Для устройства контроля положения штабелера были выбраны микроконтроллер Wemos TTGO WiFi/Bluetooth BLE с OLED-экраном 0.96 дюйма на контроллере ESP-WROOM-32 ESP32 и дальномеры VL53L1/0X;
6. Были разработаны алгоритмы работы основных функций устройства контроля положения штабелера: функция чтения данных с датчиков и функции отправки данных на облачную платформу;
7. Был разработан исходный код устройства контроля положения штабелера, который измеряет расстояние и отправляет данные на облачную платформу, на языке C++;
8. Была разработана принципиальная электрическая схема подключения микроконтроллера ESP-WROOM-32 к дальномерам VL53L1/0X;
9. Был разработан исходный код обработчика облачной платформы Rightech IoT Cloud для анализа полученных от устройства данных и определения положения штабелера на языке JavaScript версии ES6;
10. Было собрано устройство контроля положения штабелера автоматизированного склада;
11. Был разработан интерфейс мобильного приложения, который показывает актуальные координаты штабелера и его положение, если тот не находится в промежуточных позициях.
12. Апробирование показало работоспособность системы, которая в дальнейшем может быть включена в общую ГПС Denford и взаимодействовать с другими модулями ГПС.

20